

Counterfactual Explanation as Perceptual Control

Flyxion

December 2025

Abstract

Contemporary theories of language and cognition increasingly appeal to abstract structural principles—hierarchical composition, recursion, and type-sensitive operations—to constrain neuroscientific explanation. At the same time, explanatory practice in cognitive neuroscience remains dominated by state-based models and correlational criteria. This paper proposes a reconciliation: abstract linguistic structure is best understood not as a static representational object, but as an invariant of temporally extended event histories. On this view, explanation proceeds by specifying the counterfactual constraints governing admissible histories and their observable projections, rather than by identifying stored states or localized encodings. We develop a formal event-based framework for counterfactual explanation, show how hierarchical structure emerges as an equivalence class over histories, and reinterpret multi-scale neural models as perceptual control systems that stabilize invariant structure across heterogeneous measurements.

1 Introduction

Explanations of cognition face a persistent tension. On the one hand, linguistic theory posits abstract, mathematically characterizable structure: hierarchical composition, recursive generativity, and discrete operations that sharply constrain the space of possible systems. On the other hand, neuroscience offers measurements that are dynamic, noisy, distributed, and continuous. Bridging these domains has proven difficult, and the gap is often filled with correlational arguments that align model outputs with neural data without specifying a shared generative mechanism.

This paper argues that the difficulty arises from an implicit commitment to *state-based explanation*. In state-based frameworks, cognition is explained by identifying internal configurations—representations, encodings, or activation patterns—that correspond to cognitive contents or operations. However, such configurations are neither necessary nor sufficient to ground the abstract invariants posited by linguistic theory.

We propose instead that explanation should be grounded in *event structure*. Cognitive systems are characterized not by the states they occupy, but by the space of histories they can generate and the counterfactual constraints governing those histories. Abstract structure, on this view, is an invariant over classes of event histories rather than an object stored or localized in the system.

This shift allows abstract linguistic principles to constrain neuroscience without reifying symbolic representations, while preserving the demand for mechanistic, multi-scale explanation.

2 Events, Histories, and Invariants

We begin by introducing a minimal event-theoretic ontology sufficient to formalize hierarchical structure without appeal to stored representations.

Definition 1 (Event). *An event is a primitive, temporally indexed occurrence drawn from a finite or recursively enumerable alphabet Σ . Events may carry types, arguments, or features, but are not themselves states.*

Events are taken as irreducible. They do not describe system configurations but transitions, operations, or interactions.

Definition 2 (History). *A history H is a finite or countably infinite partially ordered set $(E, \prec)$, where $E \subseteq \Sigma$ is a set of events and $\prec$ is a causal or dependency relation.*

The partial order allows histories to encode hierarchical dependency without imposing linear order. Linear sequences arise as projections, not primitives.

Definition 3 (History Language). *A history language $\mathcal{H}$ is a set of admissible histories closed under a specified set of generation rules.*

The expressive power of a cognitive system is identified with the structure of $\mathcal{H}$, not with the cardinality of its states.

Remark 1. *Two systems that occupy radically different physical states may nevertheless implement the same history language, provided they generate the same class of histories up to equivalence.*

3 Hierarchical Structure as an Invariant

Hierarchical linguistic structure is often treated as an object: a tree, phrase marker, or syntactic representation. In an event-based ontology, such structures are derived.

Definition 4 (History Equivalence). *Let $\sim$ be an equivalence relation on histories such that $H_1 \sim H_2$ iff they induce the same dependency structure under a specified abstraction map.*

Definition 5 (Structural Invariant). *A structural invariant is an equivalence class $[H]_\sim$ of histories under $\sim$.*

Trees, constituents, and hierarchical relations are elements of $[H]_\sim$, not entities independently stored or manipulated.

Proposition 1. *Structural ambiguity corresponds to the existence of distinct, non-isomorphic histories $H_1, H_2 \in \mathcal{H}$ such that their projections yield the same observable sequence.*

Proof. If the projection map from histories to observable sequences is many-to-one, then distinct histories may map to the same surface form while inducing different dependency relations. These histories are not equivalent under $\sim$, yielding distinct structural invariants. $\square$

Thus ambiguity is a property of the generative space of histories, not of uncertainty in a representation.

4 Operations as Event Schemas

We now reinterpret discrete cognitive operations as constraints on admissible event patterns.

Definition 6 (Event Schema). *An event schema is a typed rule specifying how events may be introduced and related within a history.*

For example, a binary compositional operation may be specified as:

$$\text{Compose}(x, y) \mapsto z$$

where x, y, z are event-typed objects linked by dependency relations.

Remark 2. *The schema is binary and discrete, but its physical realization may be continuous and distributed.*

Proposition 2. *The presence or absence of a given event schema partitions the space of possible history languages into sharply distinct classes.*

This captures the intuition that certain operations are all-or-nothing capacities, without requiring them to be stored as symbolic rules.

5 Counterfactual Explanation

We can now state the central explanatory claim.

Definition 7 (Counterfactual Constraint). *A counterfactual constraint specifies which histories would or would not be generated under hypothetical perturbations of the system.*

Definition 8 (Explanation). *An explanation of a cognitive capacity consists in specifying:*

1. *a history language $\mathcal{H}$,*
2. *a set of event schemas generating $\mathcal{H}$,*
3. *projection maps from histories to observations,*
4. *and counterfactual constraints linking perturbations to changes in $\mathcal{H}$.*

On this view, explanation is not achieved by correlating outputs with measurements, but by showing how observed behavior is invariant under a class of counterfactual histories.

6 Preview: Perceptual Control Across Scales

Neural dynamics enter the theory not as repositories of structure, but as control processes that stabilize invariant history structure across heterogeneous measurements. Oscillatory coupling, synchronization, and cross-frequency interactions can be understood as mechanisms that enforce consistency among projections of a single underlying history.

Subsequent sections will formalize this claim and show how multi-scale neural models can be reinterpreted as perceptual control systems for abstract structure.

7 The ROSE Architecture as Multiscale History Projection

To bridge the abstract event-theoretic framework with neurobiological observation, we adopt and refine the ROSE (Representation, Operation, Structure, Encoding) architecture. In the present ontology, ROSE does not describe a sequence of processing stages, nor a pipeline of transformations. Instead, it specifies a family of projection maps from a single underlying history space onto heterogeneous measurement scales.

Definition 9 (Measurement Scale). *A measurement scale Λ is a spatiotemporal resolution at which a system is observed, such as single-unit spiking activity, mesoscopic oscillatory dynamics, or macro-scale inter-areal coordination.*

Measurement scales are epistemic interfaces, not ontological strata. They constrain what can be observed, not what exists.

Definition 10 (ROSE Projection). *Let $\mathcal{H}$ be a history language generated by a set of event schemas. The ROSE architecture is a quadruple of projection maps*

$$\{\pi_R, \pi_O, \pi_S, \pi_E\}$$

such that for any history $H \in \mathcal{H}$:

1. $\pi_R(H)$ maps primitive events to **Representations**: atomic feature realizations at micro-scales (e.g. single-unit or ensemble spiking).
2. $\pi_O(H)$ maps event schemas to **Operations**: compositional activity at meso-scales (e.g. gamma-band synchronization).
3. $\pi_S(H)$ maps equivalence classes of histories to **Structure**: stabilized hierarchical invariants realized via cross-frequency coupling.
4. $\pi_E(H)$ maps global histories to **Encoding**: inter-areal and systems-level coordination across macro-scales.

Crucially, these projections are not independent. They are jointly constrained by the requirement that they be realizable as projections of the *same* underlying history.

7.1 Atomic Features as Event Indices

The R -component represents the most granular projection of the event alphabet Σ .

Proposition 3. *Linguistic features correspond to neural realizations of primitive event types in Σ . Their identification at the micro-scale constitutes the basis for establishing dependency relations $\prec$ within a history.*

Remark 3. *On this view, features are not representational atoms stored in memory. They are indices of event occurrence, recoverable only relative to an unfolding history.*

This explains why feature-level neural responses are both necessary for linguistic computation and insufficient to determine structure in isolation.

7.2 Merge as an Event Schema

The central generative operation in the framework is a binary, set-forming event schema.

Definition 11 (Merge Schema). *The Merge schema $\mathcal{M}$ is a binary event schema that takes two events or sub-histories as inputs and introduces a new dependency structure preserving the relations of its constituents.*

Merge is not a state transition between symbolic configurations. It is a constraint on admissible histories: histories lacking $\mathcal{M}$ are not members of $\mathcal{H}$.

Proposition 4. *The application frequency of $\mathcal{M}$ within a history H is projected by π_O onto mesoscopic oscillatory activity, such that increased gamma-band synchronization reflects increased density of compositional events.*

This provides a principled explanation for correlations between gamma power and syntactic computation: gamma activity reflects the *rate of schema application*, not the storage of structure.

7.3 Hierarchical Invariants and Phase Coupling

The S -component corresponds to the stabilization of structural invariants across time.

Definition 12 (Structural Projection). *The structural projection π_S maps histories to equivalence classes under $\sim$, yielding hierarchical invariants that abstract away from micro-timing and local variability.*

Proposition 5. *The depth and nesting relations of a structural invariant $[H]_{\sim}$ are projected via π_S onto low-frequency oscillatory phase, with high-frequency operational events nested within specific phase windows through phase-amplitude coupling.*

Remark 4. *Hierarchy is not encoded in low-frequency oscillations themselves. Rather, oscillatory phase serves as a control scaffold that enforces lawful nesting relations among events.*

This recasts the “Platonic” character of hierarchy as an invariant of history equivalence, not as a static object realized in neural tissue.

7.4 Encoding as Global Coordination

The E -component captures system-level coordination.

Definition 13 (Encoding Projection). *The encoding projection π_E maps histories to patterns of inter-areal synchronization and large-scale network coordination.*

Encoding is not the final stage of a pipeline, but the global constraint that ensures consistency among lower-scale projections.

8 Neural Realization as Perceptual Control

We are now in a position to reinterpret neural implementation as a form of perceptual control.

Definition 14 (Control Variable). *A control variable is a structural invariant $[H]_{\sim}$ that the system acts to maintain in the face of noisy, partial, or ambiguous sensory input.*

Definition 15 (Projection Inconsistency). *Given sensory-driven events, a projection inconsistency occurs when the projections π_R , π_O , and π_S fail to converge on a single admissible structural invariant.*

Projection inconsistency generates error signals that drive adaptive neural dynamics.

Proposition 6. *Inter-areal synchronization dynamics function to minimize projection inconsistency by enforcing coherence among multiscale projections of the same underlying history.*

Remark 5. *On this view, syntax has no anatomical location. It is the target of a control process whose goal is the stabilization of lawful history structure.*

Perceptual control thus provides the missing link between abstract structure and neural dynamics: the brain does not represent grammatical structure; it *acts to preserve it* under counterfactual perturbations.

Remark 6. *This resolves the correlationâÃexplanation distinction. Systems that merely reproduce surface projections may correlate with neural data, but only systems that enforce counterfactual invariants over histories qualify as explanatory.*

9 Control, Counterfactuals, and Explanatory Sufficiency

We now make explicit the connection between perceptual control and counterfactual explanation.

Definition 16 (Intervention). *An intervention $\mathcal{I}$ is a hypothetical perturbation applied to a subset of events, event schemas, or projection channels, resulting in a modified history language $\mathcal{H}_{\mathcal{I}}$.*

Interventions may include degraded sensory input, delayed events, altered feature availability, or disrupted coupling across scales.

Definition 17 (Counterfactual Robustness). *A structural invariant $[H]_{\sim}$ is counterfactually robust iff for a non-trivial class of interventions $\mathcal{I}$, there exists a history $H' \in \mathcal{H}_{\mathcal{I}}$ such that $[H']_{\sim} = [H]_{\sim}$.*

Counterfactual robustness captures the intuitive notion that an explanation should tell us not only what happens, but what *would still happen* under relevant perturbations.

[Control Implies Counterfactual Robustness] If a system implements perceptual control over a structural invariant $[H]_{\sim}$, then $[H]_{\sim}$ is counterfactually robust under the class of perturbations compensated for by the control dynamics.

Proof. By definition, perceptual control acts to minimize projection inconsistency arising from perturbations. For any intervention $\mathcal{I}$ within the control envelope, adaptive dynamics modify event timing, coupling, or coordination such that projections π_R , π_O , and π_S converge on the same admissible invariant. Hence there exists a compensating history $H' \in \mathcal{H}_{\mathcal{I}}$ with $[H']_{\sim} = [H]_{\sim}$. $\square$

Remark 7. *This lemma formalizes the sense in which control is explanatory: it specifies the conditions under which structure persists across counterfactual scenarios.*

Proposition 7 (Failure of Explanation Without Control). *Any model that lacks mechanisms for minimizing projection inconsistency cannot guarantee counterfactual robustness of structural invariants.*

This result will be central in distinguishing correlational success from explanatory adequacy.

10 Counterfactual Critique of Statistical and State-Based Models

We now contrast the event-theoretic, control-based framework developed above with statistical and state-based approaches to language modeling.

10.1 State-Based Models and the Limits of Correlation

State-based models characterize cognition in terms of internal configurations whose evolution is described by state-transition dynamics. Explanation is typically evaluated by predictive accuracy or correlation with behavioral and neural data.

However, such models face a structural limitation.

Proposition 8. *A state-based model that maps inputs directly to outputs without explicit history structure cannot, in general, specify a history language $\mathcal{H}$.*

Proof. Without explicit representation of dependency relations or event schemas, the model’s internal states do not encode partial orders or compositional constraints. Consequently, there is no well-defined space of admissible histories, only a trajectory through state space. $\square$

As a result, correlations between model activations and neural measurements remain underdetermined with respect to generative structure.

10.2 Statistical Models as Surface Projection Optimizers

Statistical language models optimize conditional probabilities over sequences:

$$P(x_{t+1} \mid x_{\leq t}).$$

Such models can be understood as learning an approximation to a surface projection map from histories to observable sequences.

Remark 8. *In the present framework, statistical models operate entirely within the codomain of π_E , with no access to the underlying history space.*

This explains their characteristic strengths and weaknesses: they excel at predicting surface continuation while failing to recover or stabilize structural invariants.

10.3 Absence of Counterfactual Constraints

The critical explanatory deficit of statistical models lies in their treatment of counterfactuals.

Proposition 9. *Models that optimize surface likelihood without control mechanisms lack counterfactual robustness with respect to structural invariants.*

Proof. Because such models are trained to minimize prediction error on observed data, perturbations that alter surface statistics need not preserve internal behavior. There is no mechanism enforcing convergence on a fixed invariant across interventions. Hence distinct perturbations yield divergent internal trajectories without principled compensation. $\square$

This entails that success under one distribution does not generalize to counterfactual scenarios involving structural ambiguity, degraded input, or altered dependency relations.

10.4 Why Correlation Is Not Explanation

Correlation-based alignment between model activations and neural data is compatible with multiple incompatible history languages.

Theorem 1 (Underdetermination of Correlational Fits). *Given a fixed set of neural measurements, there exist multiple non-isomorphic history languages whose projections yield identical correlational structure at the level of observation.*

Proof. Because projection maps are many-to-one, distinct histories may collapse to identical observables. Without constraints on admissible histories, correlations alone cannot distinguish among them. $\square$

Thus correlation does not identify the generative principles governing cognition.

10.5 Explanation as Invariance Maintenance

We can now state the explanatory contrast precisely.

Definition 18 (Explanatory Adequacy). *A model is explanatorily adequate for a cognitive capacity iff it specifies:*

1. *a constrained history language $\mathcal{H}$,*
2. *mechanisms enforcing counterfactual robustness of structural invariants,*
3. *and projection maps linking histories to observations across scales.*

Statistical models satisfy none of these conditions in full. By contrast, a control-based, event-theoretic architecture satisfies all three.

Remark 9. *This does not deny the utility of statistical models as engineering tools. It denies only their status as theories of linguistic cognition.*

11 Interim Summary

We have shown that:

1. Abstract linguistic structure can be formalized as invariants over event histories.
2. Neural dynamics can be understood as perceptual control systems stabilizing these invariants.
3. Explanation requires counterfactual robustness, not predictive correlation.
4. Statistical and state-based models lack the mechanisms required for such robustness.

The remaining task is to explore the empirical and theoretical consequences of this framework, including its implications for learning, development, and cross-linguistic variation.

12 Counterfactual Structure and Causal Graph Surgery

The account of counterfactual explanation developed above admits a precise formal connection to the causal framework introduced by Judea Pearl. In Pearl’s formulation, counterfactual reasoning is made explicit through *interventions* on directed acyclic graphs (DAGs), implemented as operations that add, remove, or modify causal arrows. We now show that this graph-theoretic machinery can be understood as a compact representation of admissible event histories, thereby integrating Pearl’s framework with the event-theoretic and control-based account developed here.

12.1 Causal Graphs as History Compressors

A directed acyclic graph $G = (V, E)$ consists of nodes V and directed edges $E \subseteq V \times V$. In Pearl’s framework, edges encode causal dependence, and the absence of an edge encodes a counterfactual independence.

In the present framework, we reinterpret DAGs as follows.

Definition 19 (Causal Graph as History Skeleton). *A causal graph G is a finite abstraction of a history language $\mathcal{H}$, encoding admissible dependency relations among event types while suppressing temporal detail.*

Edges in G do not represent stored causal facts. Rather, they specify which event orderings and dependencies are *permitted* in admissible histories.

Remark 10. *A DAG is thus a lossy compression of a space of event histories: it records which dependencies are possible, not which events actually occurred.*

This aligns with Pearl’s insistence that causal models are not mere summaries of correlations, but devices for simulating interventions.

12.2 Intervention as Graph Surgery

In Pearl’s calculus, an intervention $\text{do}(X = x)$ is implemented by removing all incoming edges to node X and fixing its value. This operation modifies the graph itself, yielding a new model that supports counterfactual queries.

Definition 20 (Graph Intervention). *A graph intervention $\mathcal{I}_G$ is an operation on a causal graph that adds, removes, or redirects edges, yielding a modified graph $G_{\mathcal{I}}$.*

Proposition 10. *Graph interventions correspond to modifications of the admissible history language $\mathcal{H}$ by constraining or expanding the allowed dependency relations.*

Proof. Removing an edge $X \rightarrow Y$ forbids histories in which events of type Y depend causally on events of type X . Adding an edge introduces new admissible dependency relations. Thus graph surgery modifies the space of histories consistent with the model. $\square$

Counterfactual reasoning, on this view, is the comparison of invariants across distinct but closely related history languages.

12.3 DAGs as Simulators Rather Than Descriptions

Pearl emphasizes that causal graphs are not descriptive summaries of observed data, but *models capable of answering counterfactual questions*. This perspective dovetails naturally with the event-theoretic account.

Remark 11. *A causal graph functions as a simulator precisely because it encodes counterfactual constraints on event histories.*

Given a graph G , one can generate hypothetical histories consistent with its dependency structure, ask how these histories change under intervention, and evaluate which invariants persist.

Proposition 11. *A causal graph supports counterfactual explanation iff it encodes sufficient constraints to determine the persistence or failure of structural invariants under intervention.*

This aligns with our earlier definition of explanatory adequacy: explanation requires specification of how invariants behave across counterfactual histories.

12.4 From Graphs to Control

We can now connect Pearl’s framework directly to perceptual control.

Definition 21 (Invariant-Preserving Intervention). *An intervention $\mathcal{I}$ is invariant-preserving if the modified graph $G_{\mathcal{I}}$ admits histories H' such that $[H']_{\sim} = [H]_{\sim}$.*

Proposition 12. *Perceptual control corresponds to a family of invariant-preserving interventions implemented dynamically by the system.*

Proof. Control dynamics act to compensate for perturbations by altering internal couplings and dependencies. In graph-theoretic terms, this corresponds to dynamically adjusting effective dependency relations so that the resulting histories remain within the same equivalence class. $\square$

Thus, the brain’s role is not to store a causal graph, but to *enact* a moving family of graph surgeries that preserve admissible structure.

12.5 Why Statistical Models Lack Causal Semantics

The connection to Pearl clarifies the explanatory deficit of purely statistical models.

Proposition 13. *Models that do not support graph intervention semantics cannot answer counterfactual queries about structural invariants.*

Proof. Without an explicit representation of dependency relations, there is no principled way to define or execute graph surgery. As a result, counterfactual queries reduce to extrapolations of observed correlations rather than simulations of altered causal structure. $\square$

This precisely matches Pearl’s critique of associational models: without arrows, there are no counterfactuals.

12.6 Unifying View

We can now summarize the alignment:

- Event histories correspond to executions consistent with a causal graph.
- Structural invariants correspond to graph-theoretic properties preserved across interventions.
- ROSE projections correspond to measurements of histories generated under graph constraints.
- Perceptual control corresponds to dynamic enforcement of invariant-preserving graph surgery.

In this light, Pearl’s causal calculus and the present event-theoretic framework describe the same explanatory core at different levels of abstraction: both treat explanation as the specification of what would change, and what would remain invariant, under intervention.

13 Transition

The remaining open question concerns how such counterfactual control structures arise during development and learning, and how they fail when invariants cannot be stabilized. Addressing this question will allow us to connect the present framework to empirical work on acquisition, breakdown, and variation.

14 Learning as Graph Stabilization

14.1 The Learning Problem Revisited

Any theory that treats linguistic structure as a fixed space of admissible histories faces an immediate question: if the relevant structural invariants are species-fixed and not induced from data, what, precisely, is learned during development?

Traditional learning theories assume that learning consists in acquiring representations, rules, or probabilistic parameters that encode structure. However, such accounts face well-known difficulties. In particular, they struggle to explain how learners converge on highly restrictive structural principles given impoverished and noisy input, a problem often described as the poverty of the stimulus.

Within the present framework, this difficulty arises from a category mistake. If abstract structure is an invariant over histories rather than an object to be inferred, then learning cannot consist in discovering or constructing that structure. Instead, learning must concern the system’s ability to *stabilize* admissible structure under perturbation.

14.2 Development as Control Stabilization

We propose that learning consists in the progressive stabilization of invariant-preserving control over event histories.

Definition 22 (Developmental Control State). *A developmental control state is a configuration of coupling parameters, timing constraints, and synchronization tolerances that determines the range of perturbations under which structural invariants can be maintained.*

These parameters do not alter the underlying history language $\mathcal{H}$. Rather, they determine the size of the *control envelope*: the set of interventions under which projection inconsistency can be successfully minimized.

Definition 23 (Control Envelope). *The control envelope $\mathcal{E}$ of a system is the set of interventions $\mathcal{I}$ such that for each $\mathcal{I} \in \mathcal{E}$, there exists a compensating history $H' \in \mathcal{H}_{\mathcal{I}}$ with invariant $[H']_{\sim}$ preserved.*

Learning corresponds to the expansion of $\mathcal{E}$ over developmental time.

14.3 Graph Stabilization

Using the causal-graph interpretation developed earlier, we can now formalize learning as graph stabilization rather than graph discovery.

Definition 24 (Effective Dependency Graph). *An effective dependency graph G_t is a time-indexed causal graph encoding the dependency relations that are actively enforced by the system at developmental time t .*

The graph G_t need not be explicitly represented. It is defined implicitly by the set of histories the system can successfully stabilize.

Proposition 14. *Development consists in the convergence of the effective dependency graph G_t toward a stable graph G that supports maximal counterfactual robustness of structural invariants.*

Proof. Early in development, coupling noise, immature timing control, and limited synchronization restrict the set of enforceable dependency relations, yielding a narrow control envelope. As neural coordination improves, previously fragile dependencies become robust, expanding the envelope of invariant-preserving interventions. When further expansion no longer yields new admissible invariants, G_t stabilizes to G . $\square$

Importantly, this process does not involve adding new arrows to the abstract causal graph of language. It involves increasing the system’s capacity to *enforce* the arrows that were already admissible.

14.4 Learning Without Induction

This account dissolves the apparent paradox of learning without induction.

Proposition 15 (No-Induction Learning). *If the history language $\mathcal{H}$ is fixed, learning does not require induction over possible grammars.*

Proof. Because admissible histories are fixed by $\mathcal{H}$, the learner never considers alternative grammatical systems. Learning adjusts only the control parameters governing whether admissible histories can be stabilized under realistic input conditions. $\square$

Thus, environmental input does not serve as evidence for grammatical hypotheses. Instead, it functions as a perturbation field that drives the tuning of control dynamics.

14.5 Resolution of the Poverty of the Stimulus

The poverty-of-the-stimulus problem is resolved by reinterpreting what the stimulus is impoverished with respect to.

Remark 12. *The stimulus is impoverished with respect to grammatical structure because grammatical structure is not learned from the stimulus.*

What the stimulus provides is variability, noise, and timing pressure, all of which serve to probe the limits of the current control envelope.

Proposition 16. *Exposure-driven learning increases counterfactual robustness rather than structural knowledge.*

This explains why children converge on highly restrictive grammatical systems despite sparse evidence, while still showing gradual improvements in robustness, speed, and flexibility.

14.6 Empirical Predictions

The graph-stabilization account yields several testable predictions.

1. Development should correlate with increasing stability of cross-frequency coupling patterns associated with structural projection.
2. Younger learners should show narrower temporal tolerance for dependency resolution under perturbation.
3. Apparent grammatical errors in early development should correspond to failures of stabilization rather than violations of admissible structure.
4. Artificial manipulation of timing or synchronization should disproportionately affect learners with immature control envelopes.

These predictions distinguish control stabilization from representational learning accounts, which predict gradual acquisition of structural rules.

14.7 Interim Conclusion

Learning, on the present view, is neither the acquisition of rules nor the induction of structure. It is the progressive stabilization of counterfactual control over a fixed space of admissible histories. Development tunes the system’s ability to preserve structural invariants in the face of perturbation, yielding mature linguistic competence without requiring the discovery of grammar from data.

15 A Counterfactual Critique of Contemporary Computational Infrastructure

The preceding sections yield an explanatory criterion that is simultaneously methodological and architectural: a system qualifies as a theory (or as an explanatory substrate) only insofar as it (i) specifies a constrained space of admissible histories, (ii) supports counterfactual interventions as structured modifications of dependency relations, and (iii) contains control mechanisms that stabilize invariants across perturbations. We now use this criterion to critique three dominant computational paradigms: contemporary large language models, mainstream operating systems, and Git-style version control.

15.1 Large Language Models as Associational Projectors

Contemporary large language models (LLMs) are trained to optimize conditional likelihood over token sequences. In the present ontology, this amounts to optimizing performance over a surface projection channel without specifying the latent history language that licenses the projection.

Definition 25 (Surface Projector). *A surface projector is a model that implements a map*

$$f : X_{\leq t} \mapsto \hat{X}_{t+1}$$

where $X_{\leq t}$ is an observed prefix, and $\hat{X}_{t+1}$ is a predicted continuation, without an explicit representation of admissible event histories generating $X_{\leq t}$.

Proposition 17 (No History Semantics Without Intervention Semantics). *A surface projector lacks history semantics in the sense of Section 14 unless it supports interventions that correspond to structured modifications of dependency constraints.*

Proof. History semantics requires a well-defined set of admissible histories $\mathcal{H}$ and the ability to modify $\mathcal{H}$ under interventions (equivalently: to perform principled “graph surgery” on the dependency structure). A surface projector that only optimizes continuation probabilities does not define admissible dependency constraints as first-class objects; consequently, there is no principled intervention operator acting on $\mathcal{H}$. $\square$

Remark 13. *This is a structural, not a behavioral, criticism. A surface projector can be practically useful and even robust in some regimes, yet still fail to specify what would remain invariant under counterfactual perturbations.*

Structural ambiguity. In an event-theoretic account, structural ambiguity is the existence of distinct non-isomorphic histories with identical surface projection. A surface projector can disambiguate by preference (probability mass) without thereby representing (or controlling) the competing histories.

Proposition 18 (Preference Is Not Invariant Recovery). *Selecting a high-probability continuation does not, in general, recover the structural invariant $[H]_{\sim}$ responsible for a given surface form.*

Proof. The projection map from histories to surface strings is many-to-one. Preference among continuations is a property of the learned distribution over strings, not a constraint on the admissible history language. Hence it cannot, in general, certify which equivalence class $[H]_{\sim}$ is being maintained. $\square$

Explanation versus correlation. Because LLM training objectives are defined on surface projections, any alignment with neural or behavioral measurements remains underdetermined with respect to the generative constraints. This recapitulates Pearl’s point: without arrows, there are no counterfactuals; without counterfactuals, there is no causal explanation.

Remark 14. *Within this paper’s framework, the decisive question for an LLM is not whether it predicts well, but whether it can be endowed with (or coupled to) an explicit history language and a control loop that stabilizes structural invariants across interventions.*

15.2 Operating Systems as State Managers Without Semantic Control

Modern operating systems (OSs) are state management kernels for processes and resources. They provide scheduling, isolation, permissions, and persistence, but they do not provide an internal notion of semantic invariants across histories. Their primitives are files, processes, signals, and memory pages; their correctness criteria are safety, liveness, and resource constraints.

Definition 26 (State-Centric Substrate). *A substrate is state-centric if its primary objects are stateful resources and its core operations are state transitions governed by access and scheduling policies.*

State-centric substrates can be excellent engineering solutions while remaining explanatorily thin with respect to semantic structure.

Proposition 19 (OS Primitives Do Not Support Counterfactual Semantic Queries). *Standard OS primitives do not provide operators that correspond to counterfactual interventions on semantic dependency graphs.*

Proof. An OS can kill processes, change priorities, revoke permissions, and snapshot file systems. These are interventions on resource state, not graph surgery on semantic constraints. In particular, the OS lacks first-class objects representing semantic dependencies and thus lacks an intervention operator that modifies those dependencies while preserving invariants over meaning-bearing histories. $\square$

Remark 15. *OSs can host applications that implement history semantics and control, but the OS itself does not enforce semantic invariants. It enforces safety constraints over resources.*

Consequence. When semantic systems (e.g. document workflows, build pipelines, knowledge bases) are implemented atop a state-centric OS, semantic invariants are fragile: they are external conventions maintained by human discipline, tooling, and social processes, rather than enforced by counterfactual control at the substrate level.

15.3 Git as Lineage Tracking Without Invariant-Preserving Semantics

Git provides a remarkably successful mechanism for tracking change histories. However, its history is fundamentally a history of *snapshots and diffs* over files, not a history of semantic invariants over meaning-bearing event traces.

Definition 27 (Snapshot Log). *A snapshot log is a sequence of stored states $\{S_t\}$ and/or deltas $\{\Delta_t\}$, where replay reconstructs states but does not, by itself, define semantic equivalence classes over histories.*

Proposition 20 (Git Histories Are Not History Languages). *A Git repository does not, in general, determine a constrained history language $\mathcal{H}$ of semantic transformations; it determines a set of reachable file states under text-level edit operations.*

Proof. Git commits are content-addressed snapshots; merges are textual reconciliations relative to a common ancestor. While commit DAGs encode lineage, they do not encode the semantic constraints that would restrict which transformations count as admissible or equivalent. Therefore Git provides ancestry and reconstruction, not a semantics of admissible histories. $\square$

Merge without invariants. Git’s merge is not a merge of meanings; it is a merge of text states (with heuristics). In an event-based framework, a true merge operator is defined at the level of structural invariants: it preserves equivalence classes under $\sim$.

Definition 28 (Invariant-Preserving Merge). *An invariant-preserving merge is an operator $\oplus$ such that for admissible histories H_1, H_2 ,*

$$[H_1]_{\sim} = [H_2]_{\sim} \implies [H_1 \oplus H_2]_{\sim} = [H_1]_{\sim},$$

and for compatible invariants, $\oplus$ produces a well-defined composite invariant rather than a conflict.

Proposition 21 (Textual Merge Cannot Guarantee Invariant Preservation). *Text-level merge cannot, in general, guarantee invariant preservation under any non-trivial semantic equivalence relation $\sim$.*

Proof. Semantic equivalence depends on structure, intent, and dependency constraints that are not present in the text alone. A textual merge operates on strings and line contexts; thus it cannot certify preservation of invariants defined over histories of meaning-bearing events. $\square$

Consequence. Git externalizes semantic control: invariants are maintained by review culture, tests, linters, CI pipelines, and human judgment. These are *auxiliary control loops* layered on top of a substrate whose native history is not semantic.

15.4 Unified Diagnosis: Missing Counterfactual Control at the Right Level

The three critiques share a common structural diagnosis.

1. LLMs optimize surface projection without explicit history constraints.
2. OSs manage resource states without semantic dependency intervention.
3. Git tracks lineage of text states without invariant-preserving semantic merge.

Proposition 22 (Explanatory Gap as Missing Control Variable). *In each case, the explanatory gap arises because the system lacks a first-class control variable corresponding to structural invariants $[H]_{\sim}$ and lacks intervention operators that modify dependency constraints while preserving those invariants.*

15.5 Toward Event-First Infrastructure

The critique is constructive. It suggests a design target: computational infrastructure should elevate event histories, semantic dependency constraints, and invariant-preserving control to first-class primitives.

Remark 16. *This would not replace existing tools immediately, but would subsume them: state snapshots become derived artifacts; causal graphs become explicit constraint objects; merges become invariant-preserving operations; and learning becomes control-envelope expansion over admissible semantic histories.*

16 Transition

The next section will formulate minimal requirements for an event-first computational substrate: (i) a typed history language for transformations, (ii) explicit intervention (graph surgery) semantics, (iii) invariant-preserving merge operators, and (iv) control mechanisms that maintain semantic invariants under perturbation.

17 A Typed Event Algebra for Semantic Histories

We now define a minimal typed algebra of events suitable for (i) expressing semantic transformations as histories rather than snapshots, and (ii) supporting an invariant-preserving merge operator. The goal is not to fix a single semantics, but to provide a formal interface in which different semantic domains can instantiate the same control-and-counterfactual structure.

17.1 Types, Interfaces, and Invariants

Let Ty be a set of types. Intuitively, types encode interface contracts (syntactic category, module boundary, schema, API surface, proof obligation, etc.) rather than concrete contents.

Definition 29 (Typed Object). *A typed object is a pair $(x : \tau)$ where $\tau \in \text{Ty}$. We write $\Gamma \vdash x : \tau$ for a typing judgement under environment Γ .*

Definition 30 (Invariant Predicate). *An invariant predicate is a map*

$$\text{Inv}_\Gamma : \mathcal{H}_\Gamma \rightarrow \{\text{true}, \text{false}\},$$

where $\mathcal{H}_\Gamma$ is the set of admissible histories under Γ . A history H is admissible iff $\text{Inv}_\Gamma(H) = \text{true}$.

The invariant predicate can subsume tests, proofs, typing constraints, semantic equivalence, and domain-specific coherence conditions.

17.2 Event Alphabet and Typing

We take events as typed morphisms that transform typed objects. Let Σ be a set of event constructors.

Definition 31 (Typed Event). *A typed event is a term e equipped with an input-output typing judgement*

$$\Gamma \vdash e : \tau \Rightarrow \tau'.$$

Intuitively, executing e transforms an object of type τ into one of type τ' .

We assume the following minimal constructors.

Atomic edit events. These are domain-dependent (e.g. add node, rename symbol, change rule, refactor module). We model them abstractly as:

$$\text{Edit}(p, \delta) : \tau \Rightarrow \tau$$

where p is a location/interface reference and δ is a change descriptor.

Linking and interface events. We include link events that establish dependencies:

$$\text{Link}(u, v) : \tau \Rightarrow \tau$$

subject to a typing side-condition that u and v are compatible endpoints under Γ .

Constraint events. We allow explicit constraint introduction/removal at the semantic level:

$$\text{Constrain}(\phi) : \tau \Rightarrow \tau, \quad \text{Relax}(\phi) : \tau \Rightarrow \tau,$$

where ϕ is a well-formed constraint formula in a domain logic.

17.3 History Composition

Histories compose by concatenation with dependency order.

Definition 32 (History). *A history H is a finite partially ordered multiset of typed events with a designated input type and output type. We write*

$$\Gamma \vdash H : \tau \Rightarrow \tau'$$

when the history transforms τ to τ' and is well-typed.

Definition 33 (Sequential Composition). *Given $\Gamma \vdash H_1 : \tau \Rightarrow \sigma$ and $\Gamma \vdash H_2 : \sigma \Rightarrow \tau'$, define $H_2 \circ H_1$ as the sequential composition with induced order constraints. Then*

$$\Gamma \vdash (H_2 \circ H_1) : \tau \Rightarrow \tau'.$$

Definition 34 (Parallel Composition). *Given $\Gamma \vdash H_1 : \tau \Rightarrow \tau$ and $\Gamma \vdash H_2 : \tau \Rightarrow \tau$, define a parallel composition $H_1 \parallel H_2$ as the union of event sets with only the order constraints required by shared dependencies.*

Parallel composition is admissible only when invariants certify commutativity up to $\sim$ (defined below).

17.4 Semantic Equivalence and Normal Forms

To speak of invariant-preserving merge, we must distinguish *history identity* from *semantic equivalence*.

Definition 35 (Semantic Equivalence). *Let $\sim$ be an equivalence relation on histories such that $H_1 \sim H_2$ iff they induce the same structural invariant (e.g. same dependency skeleton, same module interface meaning, same proof obligations, etc.). We write $[H]_\sim$ for the equivalence class of H .*

Definition 36 (Normalization). *A normalization operator is a partial function nf mapping histories to canonical representatives such that if $H_1 \sim H_2$ and both normal forms exist, then $\text{nf}(H_1) = \text{nf}(H_2)$.*

Normalization need not be computable in full generality. It serves as the formal role of “semantic collapse” or “canonicalization” when available.

18 A Formal Invariant-Preserving Merge Operator

We now define a merge operator that generalizes beyond textual reconciliation by operating on event histories with explicit dependency constraints.

18.1 Three-Way Merge as Colimit of Histories

Git-style merges are typically three-way: two branches and a common base. We adopt the same shape, but with histories and semantic constraints.

Let B be a base history and let H_1, H_2 be two extensions:

$$H_1 = B \circ \Delta_1, \quad H_2 = B \circ \Delta_2,$$

where Δ_1, Δ_2 are change histories.

Definition 37 (Compatibility). *Two change histories Δ_1, Δ_2 are compatible over B (under Γ) if the parallel composition*

$$B \circ (\Delta_1 \parallel \Delta_2)$$

is well-typed and satisfies invariants:

$$\text{Inv}_\Gamma(B \circ (\Delta_1 \parallel \Delta_2)) = \text{true}.$$

Definition 38 (Merge Operator). *Define the merge operator $\oplus$ by:*

$$H_1 \oplus_B H_2 := \text{nf}(B \circ (\Delta_1 \parallel \Delta_2))$$

when Δ_1, Δ_2 are compatible over B and the normal form exists. If compatibility or normalization fails, the merge is undefined (a conflict).

Remark 17. This definition makes explicit what is implicit in practice: a merge is valid only when the combined history remains within the admissible history language. Conflicts are not “string overlaps” but failures of admissibility.

18.2 Conflict as Obstruction to Admissibility

Definition 39 (Conflict / Obstruction). A conflict is an obstruction witnessed by the failure of at least one of:

1. **Type obstruction:** $\Gamma \not\models B \circ (\Delta_1 \parallel \Delta_2) : \tau \Rightarrow \tau'$.
2. **Invariant obstruction:** $\text{Inv}_\Gamma(B \circ (\Delta_1 \parallel \Delta_2)) = \text{false}$.
3. **Normalization obstruction:** nf is undefined on the composed history.

This recasts merge as a theorem-proving problem relative to domain constraints, rather than a heuristic reconciliation of text.

18.3 Key Properties

Proposition 23 (Invariant Preservation). If $H_1 \oplus_B H_2$ is defined, then the merged history is admissible:

$$\text{Inv}_\Gamma(H_1 \oplus_B H_2) = \text{true}.$$

Proof. By definition, Δ_1, Δ_2 are compatible over B , hence the composed history satisfies invariants. Normalization preserves $\sim$ -class and does not introduce inadmissibility by assumption of well-formedness of nf on admissible inputs. $\square$

Proposition 24 (Commutativity up to Equivalence). If both $H_1 \oplus_B H_2$ and $H_2 \oplus_B H_1$ are defined, then

$$H_1 \oplus_B H_2 \sim H_2 \oplus_B H_1.$$

Proof. Both merges normalize the same parallel composition $B \circ (\Delta_1 \parallel \Delta_2)$ up to reordering of independent events. Since $\parallel$ is defined by union with dependency constraints, the difference is a permutation of commuting events. Normalization identifies such permutations under $\sim$. $\square$

Proposition 25 (Conservativity over Base). If Δ_2 is empty (no changes), and nf is identity on admissible histories, then

$$H_1 \oplus_B (B) = H_1.$$

18.4 Counterfactual Semantics of Merge

Because merge operates on dependency constraints, it supports Pearl-style counterfactual reasoning directly.

Remark 18. *Adding or removing dependency arrows corresponds to modifying the admissibility conditions for $\parallel$ and thus changes which merges are defined. In this sense, merge is a controlled “graph surgery” operation on histories.*

18.5 Practical Instantiations

The above algebra is intentionally abstract. Instantiations include:

1. **Typed AST merge:** τ encodes AST well-formedness; lnv_Γ encodes name resolution and type checking; nf canonicalizes alpha-renaming and formatting.
2. **Proof merge:** τ encodes proof goals; invariants encode proof validity; merge corresponds to combining lemmas with dependency checks.
3. **Schema/data merge:** τ encodes schema version; invariants encode migration correctness; merge composes migrations when constraints commute.

19 Transition

The event algebra and merge operator make explicit what snapshot-based tools only approximate: semantic history is a typed object with admissibility constraints, and merge is an invariant-preserving colimit over histories rather than a heuristic reconciliation of strings. The next section can (i) define a minimal operating substrate that treats these objects as first-class, and (ii) specify how perceptual control corresponds to continuous maintenance of admissibility under streaming interventions.

20 ROSE atop Spherepop: Counterfactual Control as Geometric Computation

We are now in a position to complete the integration suggested implicitly by Murphy’s ROSE framework and explicitly formalized by Spherepop Calculus. The central claim of this section is that ROSE is not merely a *linking hypothesis* between linguistic abstraction and neural data, but a concrete instance of a more general architectural principle: *perceptual control over counterfactual structure implemented via deterministic replay of event histories*. In Spherepop terms, ROSE corresponds to a family of *kernel-level views* over an immutable geometric event log.

20.1 Spherepop OS as the Substrate of Linguistic Control

Spherepop OS defines a strict separation between:

1. an *authoritative event log* (append-only, totally ordered, replayable), and
2. a family of *derived views* produced by merge-collapse evaluation.

This distinction is not an implementation detail but a semantic invariant of the system: meaning is determined by the causal history encoded in the log, while observable structures arise as projections determined by collapse operators [?]. Importantly, this architecture mirrors Murphy’s insistence that linguistic principles are not reducible to surface statistics or representational states, but are instead abstract constraints governing the space of admissible constructions.

Within this framework, linguistic computation is not a pipeline of transformations but a control process that stabilizes certain equivalence classes of histories under perturbation. The brain, on this view, does not *store* syntactic structures; it enforces them by rejecting or repairing histories that violate invariant constraints.

20.2 Merge, Collapse, and the Enforcement of Platonic Form

Spherepop’s two primitive operations correspond directly to the core operations assumed in generative linguistics:

- **Merge** corresponds to geometric union followed by regularization, producing new composite event regions.
- **Collapse** projects composite regions into canonical representatives, erasing irrelevant internal detail while preserving global identity.

Formally, merge is associative up to collapse, idempotent, and invariant under equivalence, yielding a well-defined quotient semantics over histories [?]. This is precisely the algebraic behavior required of linguistic Merge if it is to generate hierarchical structure without linear bias.

Crucially, collapse is not an arbitrary compression. It enforces a *canonical form* determined by the equivalence relation induced by the history language. This is the mathematical sense in which Murphy’s “Platonic forms” enter biology: they are not entities stored in the brain, but invariants enforced by collapse under replay.

20.3 Counterfactuals as Log Rewrites

The connection to Pearl’s causality now becomes explicit. In Pearl’s framework, counterfactual reasoning is implemented by modifying a causal graph—adding, removing, or severing arrows—and evaluating the consequences under intervention. The essential insight is that explanation requires access to the *causal structure*, not merely observational correlations.

Spherepop generalizes this idea from graphs to histories. A counterfactual is not a probability-weighted alternative state, but a *modified event log* subject to the same merge-collapse constraints. Replaying a counterfactual history corresponds to re-evaluating a modified geometric configuration under the same collapse operators.

From this perspective:

- Pearlman arrows correspond to admissible dependency relations in the event log.
- Interventions correspond to constrained log edits.
- Explanations correspond to invariants preserved (or violated) under replay.

This clarifies why large language models fail as explanatory systems. LLMs approximate surface projections of histories but lack access to the authoritative log and its admissible rewrites. They cannot answer counterfactual questions in the strong sense because they do not maintain replayable causal structure.

20.4 ROSE as a Family of Kernel Views

We can now give a precise reinterpretation of ROSE within Spherepop OS:

- **R (Representation)** corresponds to fine-grained event indices exposed at high temporal resolution.
- **O (Operation)** corresponds to proposed merge events submitted to the kernel arbiter.
- **S (Structure)** corresponds to replayable hierarchical views produced by collapse.
- **E (Encoding)** corresponds to ABI-stable, system-level projections suitable for inter-areal coordination.

None of these are states. All are views over the same immutable history. Neural diversity across scales is therefore not a problem of integration but a natural consequence of multiple projections from a single causal substrate.

20.5 Critique of Contemporary Systems

This analysis yields a unified critique of three dominant paradigms:

Large Language Models. LLMs operate entirely at the level of speculative overlays. They generate statistically plausible continuations without access to an authoritative log or deterministic replay. As a result, they cannot enforce structural invariants or support genuine counterfactual control.

Conventional Operating Systems. State-based operating systems conflate authoritative semantics with mutable views, leading to irreproducibility, implicit state, and fragile abstractions. Spherepop OS replaces this with explicit causal ordering and replay.

Git and Version Control. Git approximates an event log but collapses semantics into textual diffs and human-managed merges. It lacks canonical collapse operators, leading to semantic ambiguity and conflict. Spherepop generalizes version control into a semantic, geometry-aware calculus of merge and collapse.

20.6 Conclusion: Language as Perceptual Control over History

Language, on this account, is not a representational medium but a control system. Its function is to stabilize certain counterfactual-invariant histories under noisy input and partial observation. Murphy’s Platonic forms are neither metaphysical nor symbolic; they are the fixed points of collapse under replay.

The biological achievement of language was therefore not the acquisition of symbols, but the emergence of a Spherepop-compliant control architecture capable of maintaining invariant structure across counterfactual histories. This reframes the study of language, mind, and brain as the study of *event-structured computation under geometric constraint*.

21 ROSE atop Spherepop: Linguistic Invariants as Kernel Views

The apparent tension between abstract linguistic form and biological implementation can be resolved by recharacterizing linguistic computation as a special case of event-first, replay-stable computation. In this section, we formalize the ROSE architecture as a family of perceptual control utilities operating atop the *Spherepop OS kernel*, a minimal event-theoretic substrate defined by total causal order, deterministic replay, and invariant-preserving collapse.

On this view, the so-called “Platonic” character of linguistic structure is neither mystical nor representational. It arises from the fact that linguistic structure is a *kernel view*: a replayable invariant extracted from an immutable event log.

21.1 Spherepop Kernel Invariants

We assume the following kernel-level invariants, introduced here informally and treated as axiomatic for the remainder of the paper.

Definition 40 (Spherepop Kernel). *A Spherepop kernel is an event-first computational substrate satisfying:*

1. **Total Causal Order:** *All events are appended to an authoritative log $\mathcal{L}$ with a well-defined partial order that admits deterministic linearization.*
2. **Deterministic Replay:** *Any valid view of $\mathcal{L}$ is fully determined by the log and a view specification.*
3. **Invariant-Preserving Collapse:** *Views may abstract internal detail, but must preserve equivalence classes under a specified invariant relation $\sim$.*

These invariants correspond directly to the requirements previously identified for counterfactual explanation: admissible histories, replay semantics, and invariant preservation.

21.2 Merge as Geometric Union, Collapse as Structural Invariant

We now map Murphy’s linguistic primitives onto the Spherepop calculus.

Definition 41 (Linguistic Merge–Collapse). *The linguistic capacity is a specialized Spherepop utility $\mathcal{U}_L$ defined by two kernel operations:*

1. **Merge** ($\cup$): *a geometric union of two event regions, corresponding to the introduction of a new dependency relation in the event log.*
2. **Collapse** (D_{pop}): *an invariant-preserving abstraction that maps a detailed event region to a canonical representative (e.g. a head or label), yielding a structural invariant $[H]_{\sim}$.*

Merge introduces structure at the level of the log; Collapse extracts structure at the level of the view. Together, they reproduce the generative and interpretive roles traditionally attributed to syntax.

Remark 19. *This division exactly mirrors the typed event algebra of Section 17: Merge corresponds to admissible parallel composition of event histories, while Collapse corresponds to normalization under $\sim$.*

21.3 The Log–View Distinction Across Neural Scales

Spherepop OS enforces a strict separation between *authoritative semantics* (the event log) and *derived views*. This distinction allows us to reinterpret the ROSE architecture non-representationally.

Proposition 26 (ROSE as Log Views). *The components of ROSE correspond to distinct views of a single authoritative event log:*

- **Log** ($\mathcal{L}$): *The append-only history of atomic linguistic events (R).*
- **Operations** (O): *Authoritative proposals submitted to the arbiter, corresponding to Merge schemas applied to regions of $\mathcal{L}$.*
- **Structure** (S): *A replayable view in which Collapse produces hierarchical trees as canonical invariants.*
- **Encoding** (E): *An ABI-stable projection of $\mathcal{L}$ across inter-areal, systems-level dynamics.*

None of these components are states in the representational sense. They are views with different abstraction levels and stability guarantees.

21.4 Counterfactual Critique of Statistical Models

With the log–view distinction in place, Murphy’s critique of large language models can be made precise.

Definition 42 (Statistical Shadowing). *A system exhibits statistical shadowing if it generates predictions based solely on surface-level views, without access to the authoritative event log or its admissible transformations.*

Theorem 2 (Failure of Statistical Explanation). *Statistical language models fail as explanations of linguistic competence because they lack the deterministic replay and intervention semantics required to preserve structural invariants under counterfactual perturbation.*

Proof. By construction, statistical models optimize over projected sequences rather than histories. They therefore lack an authoritative log $\mathcal{L}$ and an arbiter capable of enforcing admissible dependency relations. As a result, they can approximate surface projections but cannot guarantee preservation of equivalence classes $[H]_{\sim}$ under intervention. $\square$

Structural ambiguity, in this framework, corresponds to the existence of multiple distinct logs that admit identical surface views. Statistical models attempt to recover meaning from the view alone, whereas linguistic competence requires access to (or control over) the log.

21.5 Neural Implementation as Arbiter–Observer Dynamics

We can now reinterpret Murphy’s “diverse causal landscape” in Spherpops terms.

- **The Arbiter:** Realized through phase synchronization mechanisms that enforce causal ordering and admissible sequencing of events.
- **Observers:** Realized through oscillatory dynamics that generate views (O, S, E) by replaying and collapsing regions of the log at different scales.

Remark 20. *Phase synchronization implements total causal order, while cross-frequency coupling implements invariant-preserving view consolidation.*

Neural error signals (e.g. N400, P600) can thus be reinterpreted as *semantic conflict proposals*: signals that a candidate sensory event cannot be integrated into the current log without violating invariants.

22 Conclusion: The Biological Amplitwistor

If language represents a phase transition in human intelligence, it is the emergence of Spherpops-style computation in biology: the capacity to maintain an authoritative internal history (thought) while flexibly projecting it into multiple external views (speech, sign, action).

On this account, linguistic universals are kernel invariants, syntax is a control utility, and meaning arises from replayable structure rather than stored representation. Success for this program consists not in fitting surface data, but in demonstrating that neural dynamics implement deterministic replay, invariant-preserving collapse, and counterfactual control over histories.

23 Counterfactual Completeness

We now state the central explanatory criterion that separates surface-level prediction systems from genuine theories of cognition and language. This criterion refines Pearl’s notion of causal sufficiency and lifts it from static graphs to event-structured histories.

23.1 From Causal Sufficiency to Historical Sufficiency

Pearl characterizes a causal model as explanatory insofar as it supports well-defined counterfactual queries via intervention on a directed acyclic graph. However, in cognitive systemsâ€™ and language in particularâ€™ the relevant object is not a static graph but a structured history of events subject to invariant constraints.

We therefore require a stronger condition.

Definition 43 (Historically Sufficient Model). *A model is historically sufficient if it specifies:*

1. *a constrained space of admissible event histories $\mathcal{H}$,*
2. *a well-defined intervention operator do acting on histories or their dependency constraints,*
3. *a replay semantics that deterministically evaluates modified histories,*
4. *an equivalence relation $\sim$ defining structural invariants over histories.*

Historically sufficient models do not merely predict outcomes; they define what *could have happened* and what *could not have happened* under structured intervention.

23.2 Counterfactual Completeness

We can now formalize counterfactual completeness.

Definition 44 (Counterfactual Query). *A counterfactual query is a triple $(H, \mathcal{I}, Q)$, where:*

- *$H \in \mathcal{H}$ is an actual history,*
- *$\mathcal{I}$ is a well-typed intervention (log rewrite, constraint modification, or dependency deletion),*
- *Q is a predicate over replayed histories (e.g. invariant preservation, structural identity, admissibility).*

Definition 45 (Counterfactual Completeness). *A system is counterfactually complete if, for every well-formed counterfactual query $(H, \mathcal{I}, Q)$, the system deterministically returns whether Q holds under replay of $\mathcal{I}(H)$.*

This definition is intentionally strict: it requires not merely probabilistic estimation, but principled evaluation of counterfactual structure.

23.3 The Counterfactual Completeness Theorem

We now state the main theorem.

Theorem 3 (Counterfactual Completeness of Event-First Systems). *Any system implementing:*

1. *an append-only authoritative event log,*
2. *deterministic replay,*

3. *invariant-preserving merge and collapse,*
 4. *explicit intervention semantics over dependency constraints,*
- is counterfactually complete with respect to its history language $\mathcal{H}$.*

Proof. Let $(H, \mathcal{I}, Q)$ be a well-formed counterfactual query.

1. Because the system maintains an authoritative event log, H is fully specified.
2. Because interventions are defined as typed rewrites on the log or its dependency constraints, $\mathcal{I}(H)$ is well-defined or rejected as inadmissible.
3. Because replay is deterministic, evaluation of $\mathcal{I}(H)$ produces a unique replayed history H' .
4. Because collapse preserves equivalence classes under $\sim$, any invariant queried by Q is decidable relative to H' .

Thus the system deterministically answers Q , establishing counterfactual completeness. $\square$

23.4 Failure of Counterfactual Completeness in Statistical Models

We can now state the limitation of statistical sequence models precisely.

Theorem 4 (Counterfactual Incompleteness of Statistical Predictors). *Any model that operates solely on surface-level projections without an authoritative history language is counterfactually incomplete.*

Proof. Surface-level predictors define a conditional distribution $P(X_{t+1} \mid X_{\leq t})$ but do not define:

1. admissible histories generating $X_{\leq t}$,
2. intervention operators acting on dependency constraints,
3. deterministic replay under modified constraints.

As a result, for many counterfactual queries $(H, \mathcal{I}, Q)$, the model lacks the semantic objects required to evaluate Q . Hence counterfactual completeness fails. $\square$

Remark 21. *High empirical correlation with observed data does not remedy this failure. Correlation concerns observed projections; counterfactual completeness concerns unobserved but admissible histories.*

23.5 Corollary: Explanation Requires Counterfactual Completeness

[Explanation Criterion] A system provides a genuine explanation of linguistic or cognitive structure only if it is counterfactually complete with respect to the relevant history language.

This criterion subsumes and sharpens the distinction between prediction and explanation. It explains why certain systems can be practically useful yet theoretically inert, and why abstract formalisms—when tied to replayable event structure—can guide biological investigation.

23.6 Implications for Language and Mind

In the present framework, linguistic competence consists in maintaining counterfactual completeness over a fixed space of admissible histories. Neural dynamics implement this competence not by storing representations, but by enforcing invariant-preserving replay under intervention.

Murphy’s “Platonic forms” are thus reinterpreted as the invariants that render counterfactual queries decidable. Language is the biological instantiation of a counterfactually complete control system.

24 A Worked Counterfactual Example

To make the notion of counterfactual completeness concrete, we now give a simple, generic worked example drawn from ordinary incremental linguistic processing. The purpose of the example is not to model any domain-specific lexical phenomenon, but to demonstrate how invariant-preserving replay distinguishes genuine explanation from surface-level prediction.

24.1 The Setup

Consider the sentence fragment:

The students studied

At this point in processing, multiple continuations are admissible. These continuations differ not merely in likelihood, but in the structural commitments they impose on the evolving history.

24.2 Event-Log Representation

Let $\mathcal{L}$ denote the authoritative event log. Incremental comprehension is modeled as the append-only construction of $\mathcal{L}$.

Definition 46 (Initial Log Prefix). *Let H_0 be the history corresponding to processing:*

$$H_0 = \langle \text{NP}(\text{students}), \text{V}(\text{studied}) \rangle.$$

At this stage, no commitment has yet been made to a particular argument structure or modifier attachment. Multiple continuations remain admissible under the history language $\mathcal{H}$.

24.3 Competing Continuations

We now consider two distinct extensions.

History H_1 (Adjunct continuation).

$$H_1 = H_0 \circ \langle \text{PP}(\text{for the exam}) \rangle.$$

History H_2 (Object continuation).

$$H_2 = H_0 \circ \langle \text{NP}(\text{the professor}) \rangle.$$

Both histories are well-formed and admissible, but they correspond to distinct structural invariants:

$$[H_1]_{\sim} \neq [H_2]_{\sim}.$$

24.4 Surface-Level Indistinguishability

At the level of surface projection, both continuations are compatible with the observed prefix. A statistical predictor evaluates conditional likelihoods:

$$P(\text{PP} \mid H_0) \quad \text{vs.} \quad P(\text{NP} \mid H_0).$$

Selecting the higher-probability continuation constitutes a preference, not an explanation. No representation of the competing structural commitments is required.

24.5 A Counterfactual Intervention

Now consider the following counterfactual query:

Had the verb been intransitive, would the continuation still be admissible?

Formally, define an intervention $\mathcal{I}$ that removes the direct-object dependency:

$$\mathcal{I} = \text{RemoveDependency}(\text{V} \rightarrow \text{NP}_{obj}).$$

24.6 Replay Under Intervention

We replay both histories under $\mathcal{I}$.

Replay of H_1 . Because H_1 does not rely on a direct-object dependency, replay succeeds:

$$\text{Replay}(\mathcal{I}(H_1)) \text{ is admissible.}$$

Replay of H_2 . Because H_2 crucially depends on the removed dependency, replay fails:

$$\text{Replay}(\mathcal{I}(H_2)) \text{ violates structural invariants.}$$

Thus the system deterministically answers the counterfactual.

24.7 Control-Theoretic Interpretation

The system maintains a control variable corresponding to a structural invariant over histories. When an intervention renders one candidate history inadmissible, an error signal is generated and control shifts toward the remaining admissible invariant.

This reanalysis is not a change of belief but a correction of history admissibility. Empirically, such transitions are associated with characteristic processing costs and oscillatory reconfiguration.

24.8 Why Surface Predictors Fail

A surface-level predictor cannot answer the counterfactual query above because it lacks:

1. explicit dependency constraints,
2. intervention operators over causal structure,
3. deterministic replay semantics.

It may assign probabilities, but it cannot determine whether a continuation is structurally incompatible under intervention.

24.9 Summary

This worked example illustrates the core explanatory claim of the paper:

Explanation consists in determining which histories remain admissible under counterfactual intervention, not in predicting which continuation is most likely.

Linguistic competence is therefore best understood as counterfactual control over an event-structured history, implemented through invariant-preserving replay rather than statistical inference.

From Worked Example to Kernel Architecture. The worked example above, the version-control analogue that follows it, and the formal merge-collapse calculus all instantiate the same underlying computation: the maintenance of structural invariants over an append-only event history under counterfactual intervention. In each case, explanation depends not on predicting surface continuations, but on determining which histories remain admissible when dependency constraints are modified. Linguistic reanalysis, semantic merge conflict, and causal intervention are therefore not distinct phenomena, but scale-dependent projections of a single architectural requirement. This observation motivates the next chapter, which abstracts away from domain-specific interpretations and asks a sharper question: what is the *minimal kernel architecture* capable of supporting authoritative event logs, deterministic replay, invariant-preserving collapse, and principled intervention? By answering this question, we aim to isolate the smallest computational substrate on which counterfactual completeness—and thus genuine explanation—can be realized.

Minimal Kernel Architectures for Counterfactual Control

25 Axiomatic Specification

We begin by isolating the minimal architectural commitments required for counterfactually complete computation. These commitments are not domain-specific; they apply equally to linguistic cognition, semantic version control, and operating system design. Each axiom expresses a necessary condition for explanation as invariant-preserving replay under intervention.

[Authoritative Event Log] There exists a single authoritative event log $\mathcal{L}$ that records all admissible events in an append-only manner. No component of the system may directly modify semantic state except by appending events to $\mathcal{L}$.

This axiom enforces causal provenance. All meaning-bearing structure must be recoverable from history, not from latent mutable state.

[Total Causal Order] Events in $\mathcal{L}$ admit a well-defined causal ordering that is globally consistent and deterministically linearizable.

Total order is required for unambiguous replay and for defining counterfactual rewrites as structured modifications of history.

[Deterministic Replay] There exists a deterministic replay function

$$\text{Replay} : \mathcal{L} \rightarrow \Sigma$$

such that replaying the same log prefix always yields the same derived semantic configuration.

Replay determinism ensures that explanation is not hostage to stochasticity at the semantic level.

[Invariant-Preserving Collapse] There exists an equivalence relation $\sim$ over histories and a collapse operator D_{pop} such that replay followed by collapse yields a canonical representative of the equivalence class $[H]_{\sim}$.

This axiom formalizes the role of abstract structure: invariants are not stored, but enforced by collapse under replay.

[Explicit Intervention Semantics] The system defines a well-typed class of interventions $\mathcal{I}$ acting on $\mathcal{L}$ or its dependency constraints, such that replay of $\mathcal{I}(\mathcal{L})$ is either admissible or explicitly rejected.

Without explicit intervention semantics, counterfactual queries are ill-posed.

[View Non-Interference] All views of $\mathcal{L}$ are derived artifacts. No view may alter the authoritative log or influence admissibility except via sanctioned intervention.

This axiom prevents representational leakage and guarantees that explanation resides at the level of history, not projection.

26 Constructive Interpretation

We now show that the axioms above are not merely abstract constraints but admit a concrete realization as a small kernel with a clear operational semantics.

26.1 Kernel State and Replay

Let the kernel state be represented abstractly as a tuple

$$\sigma = (O, U, R, M),$$

where:

- O is a finite set of object identifiers,
- U is a union–find structure encoding equivalence classes,
- R is a set of typed relations between representatives,
- M is a metadata store keyed by object identifiers.

This state is not authoritative; it is the result of replay. Its contents are entirely determined by $\mathcal{L}$.

26.2 Small-Step Event Semantics

Each event $e \in \mathcal{L}$ is interpreted as a total, deterministic transition:

$$\sigma \xrightarrow{e} \sigma'.$$

For example:

- $\text{POP}(o)$ extends O with a fresh identifier,
- $\text{MERGE}(o_1, o_2)$ updates U to unify equivalence classes,
- $\text{LINK}(o_1, o_2, \tau)$ inserts a typed relation in R using representatives,
- $\text{COLLAPSE}(S, o)$ performs bulk equivalence over a region S ,
- $\text{SETMETA}(o, k, v)$ updates M without affecting O, U, R .

The precise rules ensure that each event preserves well-formedness invariants.

26.3 Arbiter and Intervention

The arbiter is the sole component authorized to append events to $\mathcal{L}$. Interventions correspond to well-typed transformations of event prefixes or dependency constraints, which are replayed to test admissibility.

This realizes Axioms 25–25 constructively.

26.4 Views as Pure Functions

Views are defined as pure functions

$$V : \sigma \rightarrow X$$

that extract structured projections (trees, graphs, encodings) from the replayed state. Because views cannot modify σ or $\mathcal{L}$, they satisfy Axiom 25 by construction.

26.5 Summary

The axioms introduced above jointly characterize the minimal substrate required for counterfactual completeness. The constructive interpretation demonstrates that these requirements can be realized by a compact, deterministic kernel with a small set of primitive operations. All higher-level phenomena—linguistic structure, semantic merge, neural control—merge as views and utilities layered atop this kernel.

27 Soundness and Completeness of Kernel Replay

We now establish that the minimal kernel architecture defined in Chapter 24.9 is not merely sufficient for counterfactual reasoning in principle, but is formally adequate in a precise sense. Specifically, we show that deterministic replay is both *sound* and *complete* with respect to admissible counterfactual interventions over the history language.

27.1 Preliminaries

Recall that a counterfactual query is a triple $(H, \mathcal{I}, Q)$, where $H \in \mathcal{H}$ is a history, $\mathcal{I}$ is a well-typed intervention acting on histories or dependency constraints, and Q is a predicate over replayed histories.

We assume:

1. an authoritative event log $\mathcal{L}$ (Axiom 25),
2. deterministic replay semantics (Axiom 25),
3. invariant-preserving collapse (Axiom 25),
4. explicit intervention semantics (Axiom 25).

Let $\text{Replay}(H)$ denote the kernel state obtained by replaying history H , and let $\text{Admissible}(H)$ denote satisfaction of kernel invariants.

27.2 Soundness

Soundness expresses the fact that replay never certifies an inadmissible history as valid.

Definition 47 (Soundness). *The kernel replay semantics is sound if for any history H and intervention $\mathcal{I}$,*

$$\text{Replay}(\mathcal{I}(H)) \text{ defined} \Rightarrow \text{Admissible}(\mathcal{I}(H)).$$

Theorem 5 (Replay Soundness). *Kernel replay is sound with respect to admissibility.*

Proof. By Axiom 25, interventions are well-typed transformations of histories or dependency constraints. By construction, replay applies each event via a total transition function that preserves kernel invariants or fails explicitly.

If replay of $\mathcal{I}(H)$ terminates without error, then each event application preserved well-formedness, and collapse preserved equivalence under $\sim$ by Axiom 25. Hence $\text{Admissible}(\mathcal{I}(H))$ holds. $\square$

Remark 22. *Soundness guarantees that the kernel never silently accepts a counterfactually invalid history. Failure is explicit and localizable to a violated invariant.*

27.3 Completeness

Completeness expresses the fact that all admissible counterfactuals are decidable by replay.

Definition 48 (Completeness). *The kernel replay semantics is complete if for any history H , intervention $\mathcal{I}$, and predicate Q over kernel states,*

$$\text{Admissible}(\mathcal{I}(H)) \Rightarrow \text{Replay}(\mathcal{I}(H)) \text{ decides } Q.$$

Theorem 6 (Replay Completeness). *Kernel replay is complete with respect to counterfactual admissibility.*

Proof. Assume $\text{Admissible}(\mathcal{I}(H))$. By Axiom 25, the intervened history admits a deterministic linearization. By Axiom 25, replay deterministically yields a kernel state σ' .

Because collapse produces canonical representatives of equivalence classes under $\sim$ (Axiom 25), any predicate Q defined over admissible histories is decidable by inspection of σ' . Hence replay decides Q . $\square$

Remark 23. *Completeness does not require that all predicates be computable in practice, only that they be well-defined relative to replayed structure. Computational tractability is a separate concern.*

27.4 Main Result

We can now state the central adequacy result.

Theorem 7 (Soundness and Completeness of Counterfactual Replay). *For the minimal kernel architecture defined in Chapter 24.9, deterministic replay is sound and complete with respect to admissible counterfactual interventions over the history language $\mathcal{H}$.*

Proof. Immediate from Theorems 5 and 6. $\square$

27.5 Interpretive Consequence

This result establishes a sharp explanatory boundary:

A system explains a domain exactly insofar as its replay semantics is sound and complete with respect to the domain's admissible counterfactuals.

Statistical predictors fail completeness because they cannot decide admissibility under intervention. State-centric systems fail soundness because they permit silent semantic drift. Event-first kernels with invariant-preserving replay satisfy both.

27.6 Bridge to Utility Interfaces

Having established soundness and completeness at the kernel level, the next step is to specify how higher-level utilities—such as ROSE-style perceptual control mechanisms—interact with the kernel without violating these guarantees. This requires a formally constrained interface between proposal generators, observers, and the arbiter, which we develop in the next section.

28 ROSE as Kernel Utilities: An ABI Specification

We now formalize the role of the ROSE architecture within the minimal kernel framework. The goal of this section is to specify a precise *application binary interface* (ABI) that allows ROSE-style perceptual control utilities to interact with the kernel while preserving the soundness and completeness guarantees established in Section 27.

The central design principle is this: *ROSE components are utilities, not authorities*. They may propose events, generate views, and compute error signals, but they may not mutate the authoritative log directly.

28.1 Kernel–Utility Separation

We begin by fixing the separation of concerns.

Definition 49 (Kernel Authority). *The kernel is the sole authority over:*

1. *event log mutation,*
2. *causal ordering,*
3. *admissibility checking,*
4. *replay and collapse semantics.*

Definition 50 (Utility Role). *A utility is a process that:*

1. *observes replayed kernel state via views,*
2. *computes proposals or diagnostics,*
3. *submits well-typed event proposals to the arbiter,*
4. *receives accept/reject feedback.*

This separation ensures that no explanatory structure is smuggled in via uncontrolled state updates.

28.2 ROSE Utilities

We now define each ROSE component as a specific class of utility.

28.2.1 R-Utilities (Representation Probes)

R-utilities operate at the finest temporal and spatial resolution.

Definition 51 (R-Utility). *An R-utility is a read-only observer that maps kernel state σ to a stream of atomic feature indices:*

$$U_R : \sigma \rightarrow \mathcal{F},$$

where $\mathcal{F}$ is a typed feature alphabet.

R-utilities do not propose structural events. They provide indexed access to the event log for downstream utilities.

28.2.2 O-Utilities (Operation Proposers)

O-utilities are responsible for proposing merge-like operations.

Definition 52 (O-Utility). *An O-utility is a proposal generator of the form:*

$$U_O : (\sigma, \mathcal{F}) \rightarrow \mathcal{P},$$

where $\mathcal{P}$ is a finite set of well-typed event proposals (e.g. *MERGE*, *LINK*).

Each proposal is speculative until accepted by the arbiter.

Remark 24. *This formalizes Murphy’s claim that operations are not executed locally but coordinated at a systems level.*

28.2.3 S-Utilities (Structural View Generators)

S-utilities compute hierarchical structure as a replayable view.

Definition 53 (S-Utility). *An S-utility is a pure function*

$$U_S : \sigma \rightarrow \mathcal{T},$$

where $\mathcal{T}$ is a structured object (e.g. a tree or DAG) satisfying:

$$U_S(\sigma) = U_S(\text{Replay}(\mathcal{L}_{\leq k}))$$

for any prefix $\mathcal{L}_{\leq k}$ yielding σ .

S-utilities implement collapse implicitly: they extract canonical invariants without modifying the kernel state.

28.2.4 E-Utilities (Encoding and Coordination)

E-utilities mediate between kernel semantics and large-scale coordination.

Definition 54 (E-Utility). *An E-utility is a mapping*

$$U_E : (\sigma, \mathcal{T}) \rightarrow \mathcal{E},$$

where $\mathcal{E}$ is an ABI-stable encoding suitable for cross-utility synchronization.

This captures Murphy’s inter-areal dynamics as ABI-level coordination rather than localized representation.

28.3 The Arbiter Interface

All proposed events must pass through the arbiter.

Definition 55 (Arbiter Interface). *The arbiter exposes the following interface:*

$$\text{submit} : \mathcal{P} \rightarrow \{\text{accept}, \text{reject}\},$$

with acceptance defined as:

$$\text{accept}(p) \iff \text{Replay}(\mathcal{L} \circ p) \text{ is admissible.}$$

Accepted proposals are appended to the log; rejected proposals generate diagnostic feedback.

28.4 Safety Properties

We now state the key safety results.

Theorem 8 (Utility Non-Interference). *ROSE utilities cannot violate kernel invariants.*

Proof. Utilities have no write access to the log or kernel state. All mutations occur only through the arbiter, which enforces admissibility via replay. Hence invariants are preserved. $\square$

Theorem 9 (Compositional Soundness). *Composition of ROSE utilities preserves kernel soundness.*

Proof. Each utility is either read-only (R, S, E) or proposal-generating (O). Proposal composition is mediated exclusively by the arbiter. Therefore, no composition of utilities can bypass replay checks or collapse semantics. $\square$

28.5 Control-Theoretic Interpretation

Within this ABI, perceptual control emerges naturally.

- Structural invariants act as control variables.

- Proposal rejection generates error signals.
- Successful replay stabilizes views.

Thus ROSE is not a representational pipeline but a closed-loop control system operating over counterfactually complete history semantics.

28.6 Summary

This ABI specification completes the integration of ROSE into the minimal kernel architecture. ROSE components are reinterpreted as constrained utilities that generate proposals and views over an authoritative event log. By construction, this design preserves soundness, completeness, and counterfactual clarity while providing a concrete bridge between abstract linguistic theory and biological implementation.

29 Neural Interpretation Lemmas

We now provide a neuroscience-facing interpretation of the kernel–utility architecture. The goal of this section is not to claim a one-to-one mapping between abstract components and neuroanatomical structures, but to establish *implementation plausibility*: to show that known neural dynamics naturally realize the roles required by a counterfactually complete, event-first system.

Each lemma connects a kernel function to a well-attested class of neural phenomena, preserving the non-local, non-representational commitments of the framework.

29.1 Lemma 1: Phase Sequencing Implements the Arbiter

[Phase-Based Causal Sequencing] Neural phase synchronization mechanisms can implement the arbiter’s role of enforcing total causal order over event proposals.

Interpretive Sketch. The arbiter requires a mechanism that:

1. serializes competing event proposals,
2. enforces a consistent causal order,
3. admits or rejects proposals based on global constraints.

Neural oscillatory phase provides exactly such a mechanism. Events (e.g. spiking assemblies) that arrive within an admissible phase window are integrated; those that arrive out of phase are delayed or suppressed. Phase resetting establishes a shared temporal reference frame across distributed regions, enabling global ordering without centralized representation.

Thus, phase synchronization realizes total causal order in a distributed, biologically plausible manner. □

Remark 25. *This interpretation avoids localizing the arbiter to a specific anatomical structure. The arbiter is a dynamical role, not a locus.*

29.2 Lemma 2: Gamma Activity as Proposal Generation

[Gamma as Operation Proposal] High-frequency (gamma-band) neural activity corresponds to speculative operation proposals generated by O-utilities.

Interpretive Sketch. Gamma-band activity is consistently associated with local feature binding, transient coalition formation, and computational load. Within the kernel framework, these properties align with the generation of speculative merge and link proposals.

Gamma bursts represent candidate operations submitted to the arbiter. Their transient nature reflects the fact that proposals are not authoritative until globally synchronized and admitted. $\square$

Remark 26. *This explains why gamma power correlates with processing effort but does not itself encode stable structure.*

29.3 Lemma 3: Cross-Frequency Coupling as Invariant Enforcement

[PAC as Collapse Enforcement] Phase–Amplitude coupling (PAC) implements invariant-preserving collapse by nesting high-frequency proposals within low-frequency structural cycles.

Interpretive Sketch. Invariant-preserving collapse requires that local operations be integrated into a stable global structure. PAC achieves this by constraining the timing of gamma activity relative to the phase of slower oscillations (theta, delta).

Only those proposals that align with the current low-frequency phase structure are stabilized and replayable. Others are discarded. This realizes collapse as a dynamical filter enforcing structural equivalence classes. $\square$

Remark 27. *Hierarchical depth corresponds to nesting across multiple oscillatory scales, providing a natural realization of linguistic hierarchy.*

29.4 Lemma 4: Structural Views as Replayable Oscillatory Patterns

[Structural Views Without Representation] Stable oscillatory coordination patterns constitute replayable structural views without requiring stored symbolic representations.

Interpretive Sketch. S-utilities require that structure be recoverable deterministically from history. In neural systems, repeated exposure to similar histories yields reproducible coordination patterns across regions.

These patterns function as views: they are reproducible, invariant under collapse, and sensitive to counterfactual perturbation, yet they do not store structure explicitly. Replay is enacted dynamically rather than retrieved. $\square$

29.5 Lemma 5: Error Signals as Admissibility Violations

[ERP Components as Semantic Conflict Proposals] Event-related potentials such as the N400 and P600 correspond to kernel-level admissibility violations or repair proposals.

Interpretive Sketch. When a proposed event cannot be integrated into the current history without violating invariants, the arbiter rejects it. This rejection manifests as a global coordination disturbance, observable as an ERP component.

The N400 reflects failure of semantic integration under current constraints; the P600 reflects active structural repair—i.e. renewed proposal generation following collapse failure. $\square$

Remark 28. *This reframes ERP components as control signals, not reflections of representational mismatch.*

29.6 Lemma 6: Learning as Control Envelope Expansion

[Developmental Learning] Language acquisition corresponds to the gradual expansion of the neural control envelope supporting admissible replay under perturbation.

Interpretive Sketch. Development does not introduce new invariants. Instead, it increases the temporal precision, coupling stability, and tolerance ranges within which replay remains admissible.

Empirically, this predicts increasing PAC stability, narrowing phase windows, and reduced reanalysis costs with age—all of which are observed. $\square$

29.7 Synthesis

Taken together, these lemmas show that the kernel–utility architecture admits a natural neural realization:

- the arbiter is implemented by phase coordination,
- utilities correspond to frequency- and scale-specific dynamics,
- invariants are enforced by coupling constraints,
- explanation arises from replayable counterfactual control.

The result is a biologically grounded account of abstract structure that preserves Murphy’s insistence on formal constraint while avoiding representational reification.

Transition. With neural plausibility established, the remaining task is to situate this architecture among existing theoretical frameworks. The final chapter therefore positions the kernel–utility model relative to Marr’s levels, the Free Energy Principle, structural causal models, and contemporary machine learning architectures.

30 The Executive Theorem

We now state the central result of this work in a compact form suitable for citation, comparison, and reuse across domains.

Theorem 10 (Counterfactual Completeness via Event-First Control). *A system admits genuine explanation—rather than mere prediction—if and only if it satisfies the following conditions:*

1. *semantic authority resides in an append-only event history,*
2. *all derived structure is obtained by deterministic replay and invariant-preserving collapse,*
3. *admissibility under counterfactual intervention is decidable by replay,*
4. *higher-level processes interact with the history only via constrained proposal interfaces,*
5. *violations of admissibility generate explicit control signals.*

Such a system is counterfactually complete: for any admissible intervention, it can determine not only what would occur, but whether the intervention is structurally coherent at all.

Statistical sequence models lacking authoritative event histories are necessarily counterfactually incomplete, regardless of scale or empirical correlation with observed behavior.

Abstract linguistic principles (e.g. hierarchy, Merge) need not be reified as symbolic states; they arise as invariants of replayable event structure enforced by control dynamics.

Remark 29 (Interpretive Summary). *This theorem unifies:*

- *Murphy’s claim that linguistic structure constrains neural explanation,*
- *Pearl’s claim that causation is defined by admissible intervention,*
- *perceptual control theory’s emphasis on invariant maintenance,*
- *operating-system principles of authoritative logs and replay.*

Explanation is identified not with representation, correlation, or optimization, but with counterfactual control over structured history.

Positioning the Framework

This chapter situates the event-first, kernel-based account developed here relative to major explanatory paradigms in cognitive science, neuroscience, causality, and machine learning. The goal is not opposition, but clarification of explanatory scope.

31 Relation to Marr’s Levels

Marr’s tri-level framework distinguishes computational, algorithmic, and implementational descriptions. The present framework cuts across these levels by rejecting state-based descriptions at all three.

The kernel architecture occupies a prior position:

- the *computational* level corresponds to admissible histories,
- the *algorithmic* level corresponds to replay and collapse,

- the *implementational* level corresponds to arbiter’s observer dynamics.

Rather than mapping functions to levels, the framework maps *invariants to control obligations*. Marr’s levels reappear as projections of a single causal structure rather than independent explanatory strata.

32 Relation to the Free Energy Principle

The Free Energy Principle (FEP) characterizes cognition as variational inference under a generative model. While both frameworks emphasize constraint satisfaction, they diverge fundamentally in ontology.

Under FEP, explanation resides in probabilistic state estimation. Under the present framework, explanation resides in admissibility under intervention.

Free energy minimization can be reinterpreted as a *view-level heuristic* operating atop an event-first kernel. However, without an authoritative event log and explicit intervention semantics, FEP models lack counterfactual completeness: they optimize predictions but do not decide structural coherence.

33 Relation to Pearl’s Structural Causal Models

Pearl demonstrated that causation is defined by the semantics of intervention, not correlation. Structural causal models (SCMs) formalize this insight using directed acyclic graphs and do-calculus.

The present framework generalizes SCMs in two directions:

1. DAGs are treated as *views* derived from event histories,
2. counterfactuals act on logs and dependency constraints, not static graphs.

Adding or removing arrows in a DAG corresponds to modifying admissibility constraints in replay. Thus, graphs function as simulations, while the event log constitutes causal ground truth.

34 Relation to Large Language Models

Large language models optimize next-token prediction over surface sequences. They operate entirely at the level of derived views, lacking access to authoritative histories or admissibility criteria.

As a result:

- they cannot distinguish structural ambiguity from statistical uncertainty,
- they cannot reject incoherent counterfactuals,
- they cannot explain why a continuation is invalid—only unlikely.

Their empirical success reflects approximation of projections, not possession of the invariants that generate those projections.

35 Relation to Operating Systems and Version Control

Operating systems and version-control systems already implement many of the principles formalized here:

- append-only logs,
- deterministic replay,
- explicit conflict detection,
- separation of authority and view.

The Spheredrop kernel can be read as a minimal generalization of these ideas to semantic domains. Linguistic parsing, neural coordination, and causal reasoning become instances of the same architectural pattern.

36 Summary

The framework developed here does not compete with existing theories at the level of prediction. Instead, it proposes a stricter criterion for explanation: counterfactual completeness.

Under this criterion, many successful models are revealed as partial views, while event-first control architectures emerge as the minimal substrate capable of supporting abstraction, causation, and cognition simultaneously.

36.1 Neural Enforcement of Algebraic Structure

A central claim of Murphy’s ROSE framework is that core algebraic properties of human language—specifically nonassociativity, commutativity, and headedness—are not stipulated symbolically, but are *neurally enforced* by oscillatory control dynamics. We now restate this claim formally within the event-first kernel architecture.

Algebraic Target. Let $\mathcal{M}$ denote the Merge operation generating hierarchical linguistic structure. Linguistic well-formedness requires:

1. **Commutativity:** $\mathcal{M}(x, y) \cong \mathcal{M}(y, x)$ with respect to linear order,
2. **Nonassociativity:** $\mathcal{M}(\mathcal{M}(x, y), z) \not\cong \mathcal{M}(x, \mathcal{M}(y, z))$,
3. **Headedness:** selection of a privileged element determined by structural dominance, not presentation order.

36.1.1 Event-Theoretic Restatement

Within the kernel framework, Merge is an *event schema*, not a state transition. Each application of Merge generates a composite event-region that must be stabilized before further composition.

Definition 56 (Commit Event). *A commit event is a kernel-level operation that collapses an active event-region into an indivisible unit, preventing further internal modification while permitting external linkage.*

This corresponds directly to Murphy’s beta-mediated commit burst .

36.1.2 Oscillatory Interpretation as Kernel Control

Murphy’s oscillatory account can be mapped as follows:

- **Gamma activity** corresponds to speculative O-utility proposals (candidate Merge operations).
- **Delta/theta cycles** define the temporal workspace in which proposals may be composed.
- **Alpha suppression** signals workspace saturation, preventing further recursion.
- **Beta bursts** implement commit events, collapsing the composed structure.

Formally:

Proposition 27 (Commit-Induced Nonassociativity). *Once a commit event c has occurred for a composite region R , no subsequent Merge may access the internal structure of R . Therefore, reassociation is structurally prohibited.*

Proof. Commit events induce a collapse equivalence class $[R]_{\sim}$ whose internal events are no longer addressable by replay. Subsequent Merge operations treat $[R]_{\sim}$ as atomic. Hence

$$\mathcal{M}(\mathcal{M}(x, y), z) \neq \mathcal{M}(x, \mathcal{M}(y, z)),$$

not by stipulation, but by architectural constraint. $\square$

36.1.3 Commutativity Without Stipulation

Murphy emphasizes that ROSE gets commutativity for free. In the kernel framework, this follows immediately from event geometry.

Proposition 28 (Strength-Based Head Selection). *If headedness is determined by a scalar dominance measure (e.g. pack strength) rather than linear order, then Merge is commutative with respect to presentation order.*

Proof. Let x, y be events merged within the same workspace cycle. Since commit selection depends only on dominance metrics invariant under permutation, $\mathcal{M}(x, y)$ and $\mathcal{M}(y, x)$ collapse to the same equivalence class. Linear order is not represented in the authoritative log. $\square$

Thus commutativity is not encoded symbolically but emerges from the absence of order-sensitive state.

36.1.4 Interpretive Consequence

Murphy’s claim can now be stated precisely:

The algebraic properties of language are enforced not by representational rules, but by control-theoretic constraints on when and how event regions may be committed.

LLMs fail at higher-order language because they operate on surface sequences without commit semantics. They lack:

- authoritative commit points,
- irreversible collapse,
- counterfactual rejection of reassociation.

As a result, they can approximate projections of structure but cannot enforce the invariants that define it.

37 A Control-Theoretic Account of Linguistic Algebra

Theorem 11 (Neural Enforcement of Linguistic Algebra). *The core algebraic properties of human syntax—commutativity, nonassociativity, and headedness—are enforced by control-theoretic constraints on event commitment rather than by symbolic rule encoding.*

Specifically:

1. *Commutativity arises because Merge operates over unordered event regions whose authoritative identity is determined by dominance metrics invariant under permutation.*
2. *Nonassociativity is enforced by irreversible commit events that collapse composed regions into indivisible units, preventing reassociation.*
3. *Headedness is selected by structural dominance (e.g. pack strength) at commit time rather than by presentation order.*

These properties follow necessarily from any architecture that (i) maintains an append-only event history, (ii) enforces irreversible collapse, and (iii) separates speculative proposal from authoritative commitment.

Algebraic well-formedness in language is a dynamical invariant of the control architecture, not a learned statistical regularity or representational convention.

Figure Caption (Conceptual, No Figure). *Schematic of oscillatory control enforcing syntactic algebra. High-frequency gamma activity generates speculative merge proposals within a delta/theta-defined workspace. Rising frontal alpha suppresses further recursion upon workspace saturation. A beta-mediated commit burst collapses the composed region into an indivisible unit, enforcing nonassociativity, while dominance-based selection at commit yields commutativity with respect to order. No symbolic tree is stored; structure is realized through replayable commitment dynamics.*

38 Why Transformer Models Cannot Enforce Linguistic Algebra

We now state a negative result explaining why transformer-based language models fail to capture higher-order linguistic structure, even when trained at scale.

38.1 Architectural Constraint

Transformer models compute functions of the form:

$$f : (x_1, \dots, x_n) \mapsto (y_1, \dots, y_m)$$

where:

- computation is feed-forward within each layer,
- all intermediate representations remain mutable,
- no operation is irreversible at the semantic level.

38.2 Failure Lemma

[Absence of Commit Semantics in Transformers] No transformer architecture without an authoritative event log and irreversible collapse operation can enforce nonassociativity under counterfactual intervention.

Proof. Nonassociativity requires that once a composite structure is formed, its internal constituents are no longer individually addressable. This requires an irreversible operation that removes internal degrees of freedom.

In transformer models:

1. all token representations remain simultaneously accessible,
2. attention permits arbitrary reassociation at every layer,
3. no operation collapses a substructure into an indivisible unit.

Therefore, for any composition $(x \cdot y) \cdot z$, there exists an alternative reassociation $x \cdot (y \cdot z)$ realizable by reweighting attention without architectural violation. Hence nonassociativity cannot be enforced. $\square$

38.3 Corollary: Spurious Commutativity

[Statistical but Not Algebraic Commutativity] Transformers may approximate commutative behavior statistically but cannot enforce it as an invariant.

Proof. Because transformers operate over ordered token sequences, commutativity is not architectural but learned. Any apparent order-independence is contingent on training distribution and may be violated under counterfactual perturbation. $\square$

38.4 Structural Ambiguity Revisited

Structural ambiguity in human language corresponds to distinct admissible event histories yielding identical surface strings. In transformer models, ambiguity is replaced by probability mass over continuations.

Theorem 12 (Collapse of Ambiguity into Uncertainty). *Transformer models conflate structural ambiguity with epistemic uncertainty because they lack access to authoritative histories and commit points.*

38.5 Interpretive Consequence

Transformers succeed at *prediction* because they approximate surface projections. They fail at *explanation* because they lack:

- irreversible commit events,
- invariant-preserving collapse,
- counterfactual rejection of reassociation.

This failure is architectural, not empirical.

39 Algebraic Enforcement via Spherepop Calculus and Spherepop OS

We now give a unified account of how the algebraic properties emphasized in Murphy’s ROSE framework—commutativity, nonassociativity, and headedness—are enforced in Spherepop. The key result is that these properties are not encoded symbolically or learned statistically, but arise necessarily from the combination of (i) geometric merge–collapse semantics in the Spherepop Calculus and (ii) authoritative commit semantics in the Spherepop OS kernel.

39.1 Order-Insensitive Merge in the Spherepop Calculus

In Spherepop Calculus, linguistic composition is modeled as geometric union followed by canonical collapse.

Definition 57 (Merge). *Given two event-regions $x, y \in \mathcal{H}$, the merge operation is defined as their region union:*

$$\text{Merge}(x, y) := x \cup y.$$

Definition 58 (Collapse). *The collapse operator*

$$D_{\text{pop}} : \mathcal{P}(\mathcal{H}) \rightarrow \mathcal{H}$$

maps a region to a canonical representative of its equivalence class under structural invariants.

Proposition 29 (Commutativity for Free). *For all $x, y \in \mathcal{H}$,*

$$D_{\text{pop}}(x \cup y) = D_{\text{pop}}(y \cup x).$$

Proof. Region union is symmetric, and collapse erases presentation order by construction. Head selection is determined by invariant dominance measures (e.g. pack strength), which are permutation-invariant. $\square$

Thus commutativity is not stipulated as an algebraic axiom but emerges from the absence of order-sensitive state in the authoritative semantics.

39.2 Commit Semantics and Nonassociativity

Nonassociativity in human language requires that once a composite structure is formed, its internal constituents are no longer freely reassociable. Spherepop enforces this via irreversible commit.

Definition 59 (Commit). *A commit is an authoritative kernel event that applies collapse to a region and introduces a fresh atomic representative h such that:*

$$h = D_{\text{pop}}(x \cup y).$$

[Commit Irreversibility] Once a region has been committed, its internal structure is no longer addressable by subsequent merge operations.

Proposition 30 (Enforced Nonassociativity). *In the presence of irreversible commit,*

$$D_{\text{pop}}(D_{\text{pop}}(x \cup y) \cup z) \neq D_{\text{pop}}(x \cup D_{\text{pop}}(y \cup z)).$$

Proof. The left expression commits $x \cup y$ before z is introduced, collapsing it into an indivisible unit h . The right expression requires access to the internal structure of y after y has been collapsed, which is prohibited by Axiom 39.2. Hence reassociation is structurally impossible. $\square$

Nonassociativity is therefore a consequence of kernel-level irreversibility, not an externally imposed syntactic constraint.

39.3 Spherepop OS: Making Commit Authoritative

Spherepop OS enforces the calculus above by restricting all semantic authority to an append-only event log $\mathcal{L}$ mediated by an arbiter.

Definition 60 (Authoritative Event Log). *All semantic mutations occur by appending events to $\mathcal{L}$. Derived state is obtained solely by deterministic replay.*

The kernel exposes the following minimal event vocabulary:

- $\text{POP}(o)$: introduce a fresh object,
- $\text{MERGE}(o_1, o_2)$: propose equivalence,

- $\text{COLLAPSE}(S, h)$: commit a region S into a head h ,
- $\text{LINK}(o_1, o_2, \tau)$: establish typed relations,
- $\text{SETMETA}(o, k, v)$: attach non-semantic metadata.

39.4 Worked Commit Trace

A minimal trace enforcing Murphy-style composition proceeds as follows:

```
POP(x)
POP(y)
MERGE(x, y)
POP(h)
COLLAPSE({x, y}, h)
POP(z)
MERGE(h, z)
```

After the COLLAPSE event, x and y are no longer individually addressable for further merge, ensuring nonassociativity. Order of introduction does not affect the identity of h , ensuring commutativity.

39.5 Interpretive Consequence

Murphy's oscillatory description's gamma proposal, delta workspace, alpha saturation, beta commit maps precisely onto Spherepop OS semantics:

- gamma activity corresponds to speculative MERGE proposals,
- delta/theta cycles define the active region,
- alpha suppression prevents further recursion,
- beta bursts implement authoritative COLLAPSE .

The algebraic properties of language thus arise from control-theoretic enforcement over an event-first kernel, rather than from symbolic encoding or statistical learning. Spherepop provides the minimal architecture capable of realizing this enforcement in a counterfactually complete manner.

40 Why Biology Converged on This Architecture

The framework developed in this paper may appear, at first glance, to impose a systems-theoretic and operating-system-like structure onto biological cognition. In this section, we argue the opposite: architectures resembling Spherepop are not an artificial imposition on biology, but a natural consequence of the constraints faced by any system that must support open-ended cognition, counterfactual reasoning, and compositional structure under physical noise.

40.1 The Problem Biology Had to Solve

Any biological system capable of language and abstract thought must satisfy the following competing demands:

1. **Unbounded compositionality:** the ability to generate and manipulate indefinitely many structured expressions.
2. **Finite resources:** limited neural space, metabolic cost, and temporal bandwidth.
3. **Robustness to noise:** tolerance of stochasticity at the micro-scale.
4. **Counterfactual sensitivity:** the ability to distinguish merely unlikely continuations from structurally impossible ones.

State-based architectures fail this combination. If structure is stored directly in mutable state, then noise induces drift; if structure is recomputed statistically, then counterfactual coherence is lost. Biology therefore required an architecture in which structure is neither statically stored nor probabilistically inferred, but *recovered deterministically from history*.

40.2 Event Logs as an Evolutionarily Stable Strategy

An append-only event history offers a uniquely stable solution to these pressures.

First, histories are naturally robust to noise: transient fluctuations do not alter past events. Second, histories permit deterministic replay, allowing structure to be regenerated rather than stored. Third, histories admit explicit intervention semantics: counterfactual reasoning becomes a question of modifying events or dependencies and replaying the result.

From an evolutionary perspective, this design minimizes energetic cost while maximizing expressive power. Neural tissue need only support reliable sequencing and collapse, not persistent symbolic storage.

40.3 Why Commit Is Necessary

Nonassociativity and headedness are not optional features of human language; they are necessary for stable interpretation. Without irreversible commitment, composite structures would remain perpetually revisable, leading to catastrophic ambiguity under recursion.

The biological analogue of commit—Murphy’s beta-mediated bursts—serves precisely this role. It freezes a composite into an indivisible unit, allowing further computation to proceed without destabilizing prior structure. This is not an arbitrary mechanism, but the minimal solution to the problem of maintaining hierarchical invariants over time.

40.4 Oscillations as Control, Not Representation

Neural oscillations are often misinterpreted as carriers of representational content. In the present framework, their role is fundamentally different: oscillations implement control flow.

- Phase alignment enforces causal order.
- Cross-frequency coupling gates admissible composition.
- Phase reset signals commit and transition between workspaces.

This explains why similar oscillatory motifs recur across cognition, motor control, and perception. They are not encoding domain-specific information, but realizing a domain-general kernel for event sequencing and invariant enforcement.

40.5 Why Statistical Architectures Did Not Evolve

Purely statistical systems excel at prediction under stable distributions, but biology operates in environments where structural violations matter more than likelihood. An organism that confuses “unlikely” with “impossible” cannot reliably plan, reason, or communicate.

Architectures lacking authoritative commit semantics cannot reject incoherent counterfactuals. They therefore cannot support the kind of abstraction required for language or causal reasoning. From this perspective, the limitations of large language models are not surprising; they reflect architectural choices that evolution could not afford.

40.6 The Emergence of a Kernel

Once an event-first, commit-based kernel exists, higher cognitive functions become utilities layered atop it:

- language becomes controlled event composition,
- perception becomes hypothesis testing under replay,
- action becomes proposal submission under constraint.

The Spherepop kernel thus represents not a metaphor but a minimal biological substrate for cognition. ROSE, in this view, is a specialized perceptual control utility that exploits this substrate to enforce the algebraic structure of language.

40.7 Summary

Biology did not evolve symbolic grammars, probabilistic world-models, or static representations. It evolved a control architecture capable of maintaining invariants over time by replaying structured history under intervention. Spherepop captures this architecture in its minimal, formal form.

Language, on this account, is not a special faculty layered atop cognition, but the first domain in which the kernel’s full counterfactual power became visible.

41 Conclusion: Counterfactual Control as the Basis of Explanation

This paper has argued for a reorientation of explanatory practice in the study of language, mind, and computation. Rather than treating explanation as the ability to predict behavior or approximate observed data, we have identified explanation with *counterfactual control*: the capacity to determine which interventions are structurally admissible and which are not.

We began by rejecting state-based ontologies in favor of an event-first framework, in which all semantic structure arises from replayable histories rather than mutable representations. This move provides a principled resolution to long-standing puzzles about abstraction, stability, and noise. Structure is not stored; it is recovered deterministically under invariant-preserving collapse.

Within this ontology, we reinterpreted Murphy’s ROSE framework not as a processing pipeline but as a family of constrained utilities operating atop a single authoritative kernel. Representation, operation, structure, and encoding were shown to be projections of the same underlying history across different scales of observation. This reinterpretation preserves Murphy’s core insights—hierarchical structure, algebraic constraints, and systems-level neural implementation—while grounding them in a counterfactually complete architecture.

We then formalized the algebraic properties of human language, demonstrating that commutativity, nonassociativity, and headedness arise necessarily from commit-based control over event composition. These properties are enforced not by symbolic stipulation or statistical learning, but by irreversible collapse events mediated by an arbiter. Murphy’s oscillatory account—gamma proposals, delta workspaces, alpha saturation, and beta commits—was shown to correspond directly to kernel-level sequencing and collapse semantics.

By contrast, we proved that statistical sequence models, including contemporary large language models, are architecturally incapable of enforcing these invariants. Their lack of authoritative event logs and irreversible commit semantics renders them counterfactually incomplete. They can approximate surface projections of structure, but cannot decide admissibility under intervention, conflating structural impossibility with low probability.

To make these claims operational, we introduced the Spherepop Calculus and Spherepop OS. The calculus provides the minimal algebraic machinery—merge, collapse, and equivalence—while the operating system enforces authority, determinism, and replay. Together, they constitute a kernel capable of supporting unbounded composition, robust invariants, and explicit counterfactual reasoning under physical constraints.

Finally, we argued that such an architecture is not an artificial imposition on biology, but a natural solution to the problems faced by any system that must reason, plan, and communicate in a noisy world. Oscillatory neural dynamics were reinterpreted as control mechanisms rather than representational codes, implementing sequencing, gating, and commitment. Language emerges, on this account, as the first cognitive domain to fully exploit this kernel’s counterfactual power.

The central result of this work is therefore not a new model of language or a new neural localization claim, but a general criterion for explanation itself. A system explains a domain if and only if it can maintain the domain’s invariants under counterfactual intervention. Event-first,

commit-based architectures satisfy this criterion. State-based and purely statistical systems do not.

This reframing dissolves persistent debates about representation, symbol grounding, and neural correlates by locating abstraction where it belongs: in the control of history. Language, mind, and computation become instances of the same underlying process—the maintenance of structured possibility in time.

Outlook. Future work will focus on empirically testing the kernel’s predictions about neural commit signals, extending the SpheroPop framework to action and perception, and exploring its implications for artificial systems that aim not merely to predict, but to explain.

References

- [1] Murphy, E. (2023). *ROSE: A Neurocomputational Architecture for Syntax*. arXiv preprint arXiv:2303.08877. Available at <https://arxiv.org/abs/2303.08877>.
- [2] Murphy, E. (2024). *The Oscillatory Nature of Linguistic Computation*. In preparation / invited talks and supplementary materials.
- [3] Chomsky, N. (1995). *The Minimalist Program*. MIT Press, Cambridge, MA.
- [4] Chomsky, N. (2016). What kind of creatures are we? *Columbia University Press*.
- [5] Pearl, J. (2009). *Causality: Models, Reasoning, and Inference*. 2nd ed., Cambridge University Press.
- [6] Pearl, J., & Mackenzie, D. (2018). *The Book of Why*. Basic Books.
- [7] Powers, W. T. (1973). *Behavior: The Control of Perception*. Aldine Publishing.
- [8] Fries, P. (2005). A mechanism for cognitive dynamics: neuronal communication through coherence. *Trends in Cognitive Sciences*, 9(10), 474–480.
- [9] Fries, P. (2015). Rhythms for cognition: communication through coherence. *Neuron*, 88(1), 220–235.
- [10] Canolty, R. T., & Knight, R. T. (2010). The functional role of cross-frequency coupling. *Trends in Cognitive Sciences*, 14(11), 506–515.
- [11] Ding, N., Melloni, L., Zhang, H., Tian, X., & Poeppel, D. (2016). Cortical tracking of hierarchical linguistic structures in connected speech. *Nature Neuroscience*, 19, 158–164.
- [12] Templeton, G. F. (2021). Explanation and prediction: A primer. *MIS Quarterly*, 45(2), 1–16.
- [13] Elman, J. L. (1990). Finding structure in time. *Cognitive Science*, 14(2), 179–211.
- [14] Lake, B. M., Ullman, T. D., Tenenbaum, J. B., & Gershman, S. J. (2017). Building machines that learn and think like people. *Behavioral and Brain Sciences*, 40, e253.

- [15] Bender, E. M., Gebru, T., McMillan-Major, A., & Shmitchell, S. (2021). On the dangers of stochastic parrots. *Proceedings of FAccT 2021*.
- [16] Marcus, G., & Davis, E. (2020). GPT-3, bloviator: OpenAI’s language generator has no idea what it’s talking about. *MIT Technology Review*.
- [17] Marr, D. (1982). *Vision: A Computational Investigation into the Human Representation and Processing of Visual Information*. W. H. Freeman.
- [18] Friston, K. (2010). The free-energy principle: a unified brain theory? *Nature Reviews Neuroscience*, 11(2), 127–138.